Observabilidade e SRE com OpenTelemetry — Evidências do laboratório

14/11/2026 · João Micucci e Rafael Serpa

Identificação

A Notas da Nuvem, implantada no laboratório da Aula 10 (CloudFront, ALB, ECS Fargate e Aurora Serverless v2), passou a ser observável de ponta a ponta com OpenTelemetry. As 7 evidências da apostila de Observabilidade e SRE e os 4 extras combinados com o professor estão documentados abaixo, em 34 prints tirados em 14/11/2026 e no log do terraform destroy.

Item	Detalhe
Disciplinas	Arquitetura de Software Nativa em Nuvem · Segurança para Computação em Nuvem
Instituição	CESAR School · 2026.2
Professor	Prof. MSc. Vinicius Siqueira
Atividade	Observabilidade e Engenharia de Confiabilidade com OpenTelemetry (continuação do laboratório da Aula 10)
Região AWS	us-east-1

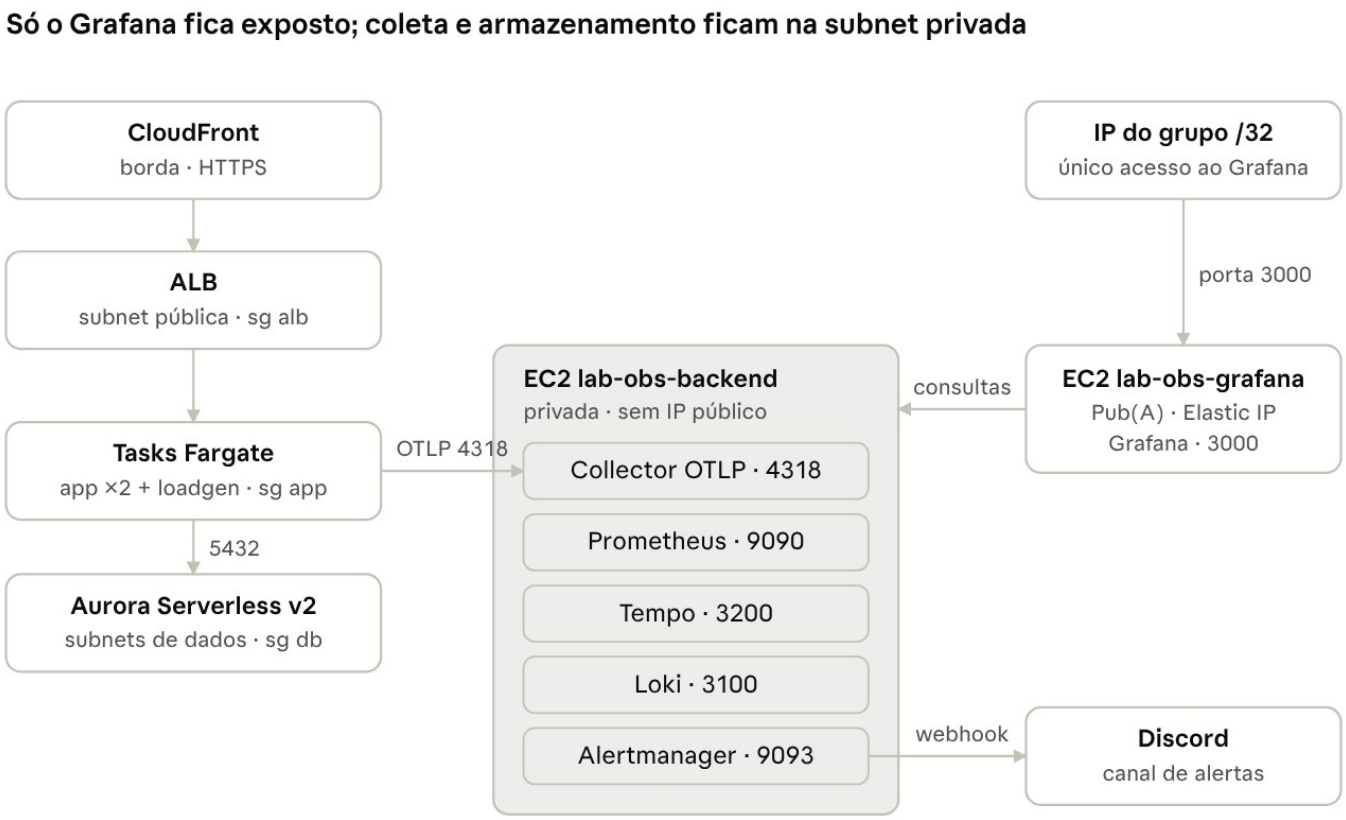
Integrantes

- João Henrique Micucci Mendes
- João Pedro Mamede
- João Pedro Moraes
- Caio Guimarães
- Kerry
- Mateus Lustosa
- Rafael Serpa

O Account ID, o IP público do Grafana e o IP autorizado no security group foram tarjados nos prints, conforme a política PO-CESAR-021.

Arquitetura

A aplicação, o banco e a borda continuam como na Aula 10. A stack de observabilidade saiu de um container único numa EC2 pública e passou para duas camadas: o Grafana numa EC2 pública, com porta restrita, e coleta e armazenamento numa EC2 sem IP público.



Arquitetura da observabilidade em duas camadas · VPC labnuvem, us-east-1

As tasks da aplicação e do loadgen enviam traces, métricas e logs via OTLP para o Collector, que distribui cada sinal ao seu backend. O Grafana consulta os backends pelo IP privado, e o Alertmanager notifica o Discord. Da internet, só o IP do grupo alcança a porta 3000.

Mapa de requisitos

Os 13 requisitos têm evidência. A limpeza (7) está comprovada pelo `terraform destroy`, que removeu tudo, inclusive um repositório ECR que precisou ser apagado à mão.

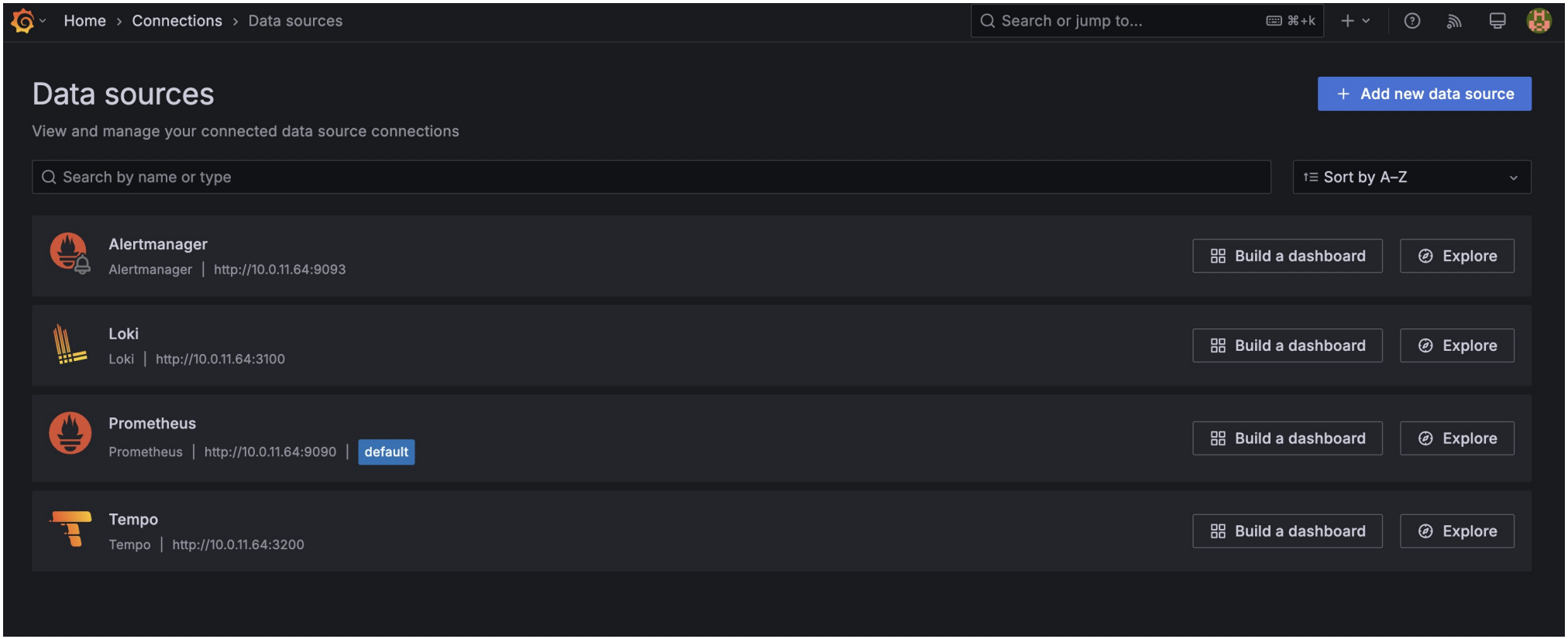
N°	Requisito	Como foi atendido	Prints
1	Data sources Prometheus, Tempo e Loki	Provisionados no Grafana, mais o Alertmanager, todos apontando para o IP privado do backend	E01-a, E01-b
2	Nova revisão com imagem OTel e variáveis `OTEL_*`	Revisão `lab-test:6`, imagem `app-2.2.0`, 7 variáveis da apostila + 2 de identidade; `DB_*` e segredos mantidos	E02-a, E02-b
3	Service com 2 tasks na nova revisão	`lab-service` com 2/2 tasks em `lab-test:6`, em us-east-1a e us-east-1b	E03-a, E03-b
4	Trace de `POST /notas` até o `INSERT`	Trace com o span `INSERT`; consulta parametrizada; tasks distinguidas por `host.name`; log ligado ao trace	E04-a a E04-e
5	Painel com SLI de disponibilidade e p95	Dashboard *Notas da Nuvem - SLI* provisionado	E05-a, E05-b
6	SLI caindo e trace com erro	Regra 5432 removida do sg `db`; SLI caiu a 58,33% e p95 subiu a 7,37 s	E06-a a E06-g
7	EC2 Terminated	`terraform destroy`: 98 de 99 recursos na 1ª execução; ECR apagado à mão; 2ª execução sem pendências	E07-a + log
8	Grafana em EC2 separada, em subnet pública	`lab-obs-grafana` na `Pub(A)` com Elastic IP; `lab-obs-backend` na `PrivateFargate(A)`, sem IP público	E08-a a E08-d
9	Portas restritas	3000 só de um IP `/32`; portas do backend só	E09-a, E09-b

N°	Requisito	Como foi atendido	Prints
		dos security groups `app` e `obs-grafana`	
10	OTLP, Tempo, Loki e Prometheus na subnet privada	5 containers na EC2 privada, imagens espelhadas no ECR	E10-a
11	Alertmanager integrado ao Grafana	Regras no Prometheus; Alertmanager como data source e contact point do Discord visível no Grafana	E11-a, E11-b
12	Integração com o Discord	Alertas FIRING e RESOLVED entregues no canal	E12-a, E12-c, E12-d
13	Tasks geradoras de tráfego	Service `notas-loadgen` instrumentado, chamando a app pelo domínio público	E13-a a E13-c

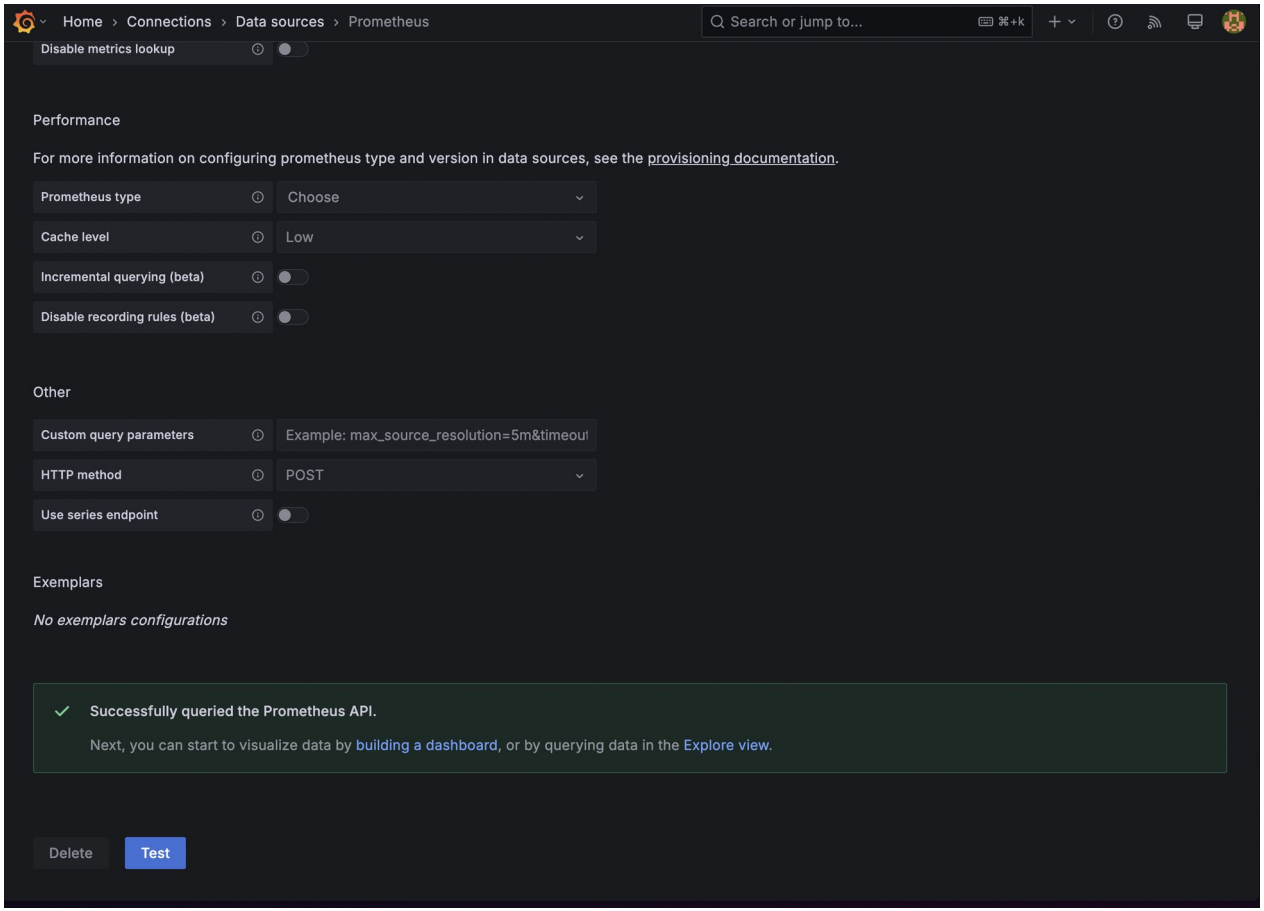
Evidências da apostila (1 a 7)

Evidência 1 — Data sources do Grafana

O Grafana tem os quatro backends provisionados: Prometheus (métricas), Tempo (traces), Loki (logs) e Alertmanager. Todos apontam para `10.0.11.64`, o IP privado da EC2 de backend, e o teste do Prometheus confirma que a EC2 pública alcança a subnet privada. A senha do admin é aleatória e fica no Secrets Manager, em vez do padrão admin/admin.



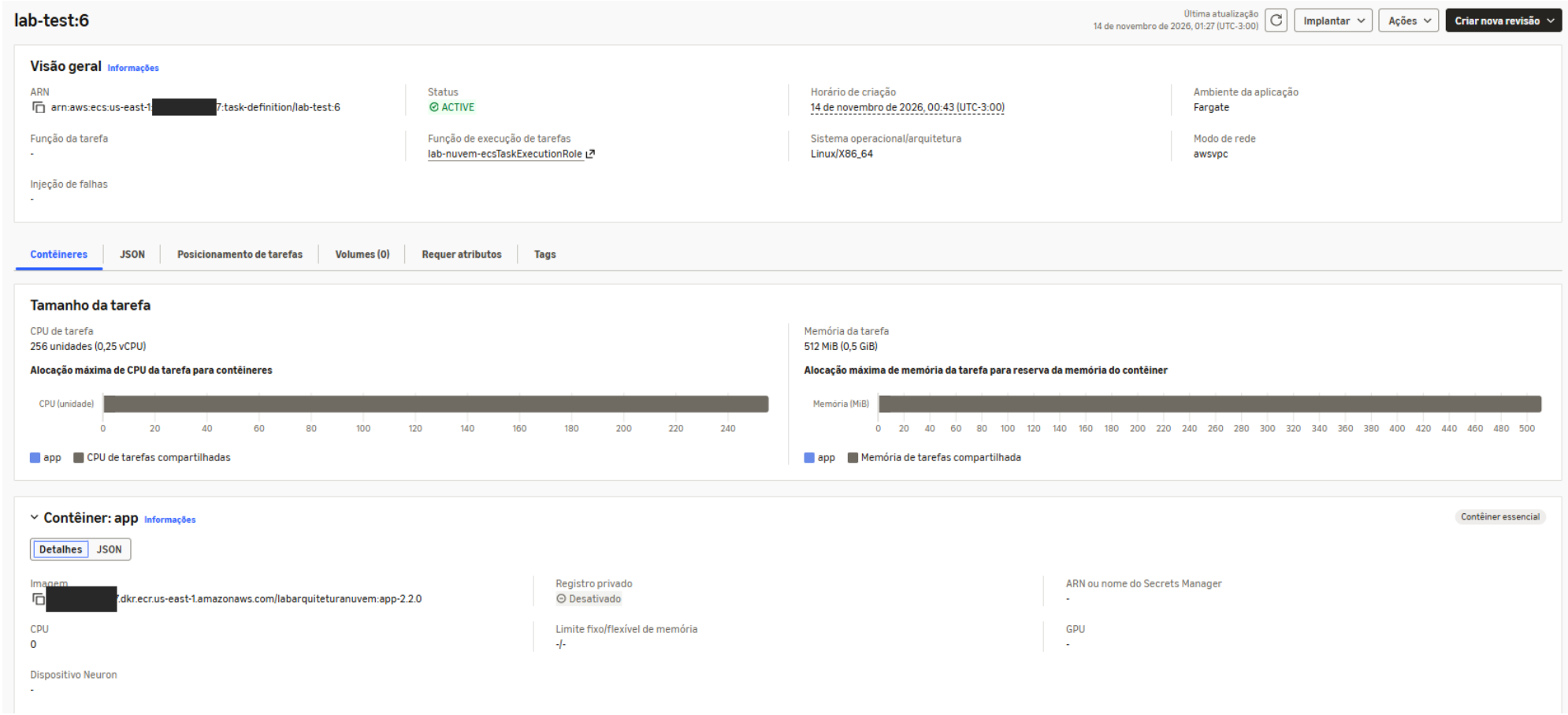
E01-a — Connections > Data sources com Alertmanager, Loki, Prometheus e Tempo.



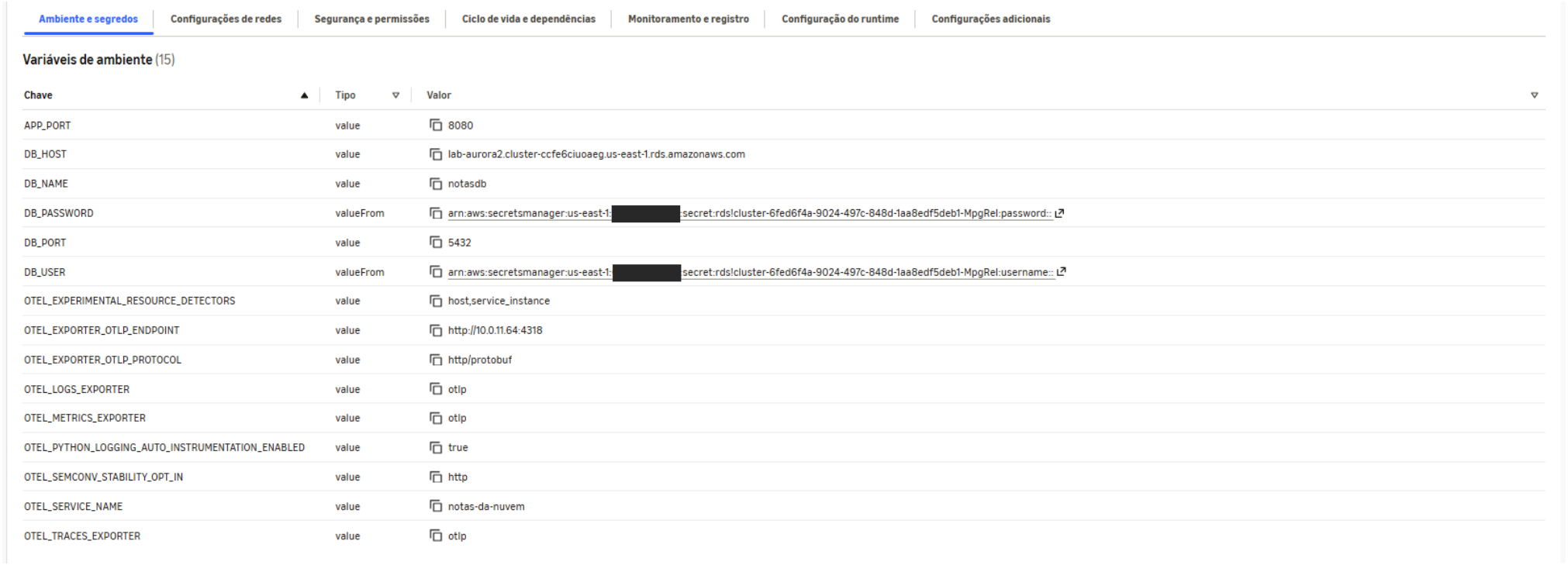
E01-b — Prometheus > Test: "Successfully queried the Prometheus API".

Evidência 2 — Nova revisão da task definition

A revisão `lab-test:6` usa a imagem `app-2.2.0`, construída com o SDK OpenTelemetry Python 1.45.2 e as instrumentações 0.66b2. Além das 7 variáveis da apostila, entraram duas de identidade: `OTEL_EXPERIMENTAL_RESOURCE_DETECTORS=host,service_instance`, que separa as tasks, e `OTEL_SEMCONV_STABILITY_OPT_IN=http`, que fixa os nomes estáveis das métricas HTTP. As variáveis `DB_*` continuaram, com usuário e senha vindos do Secrets Manager (`valueFrom`).



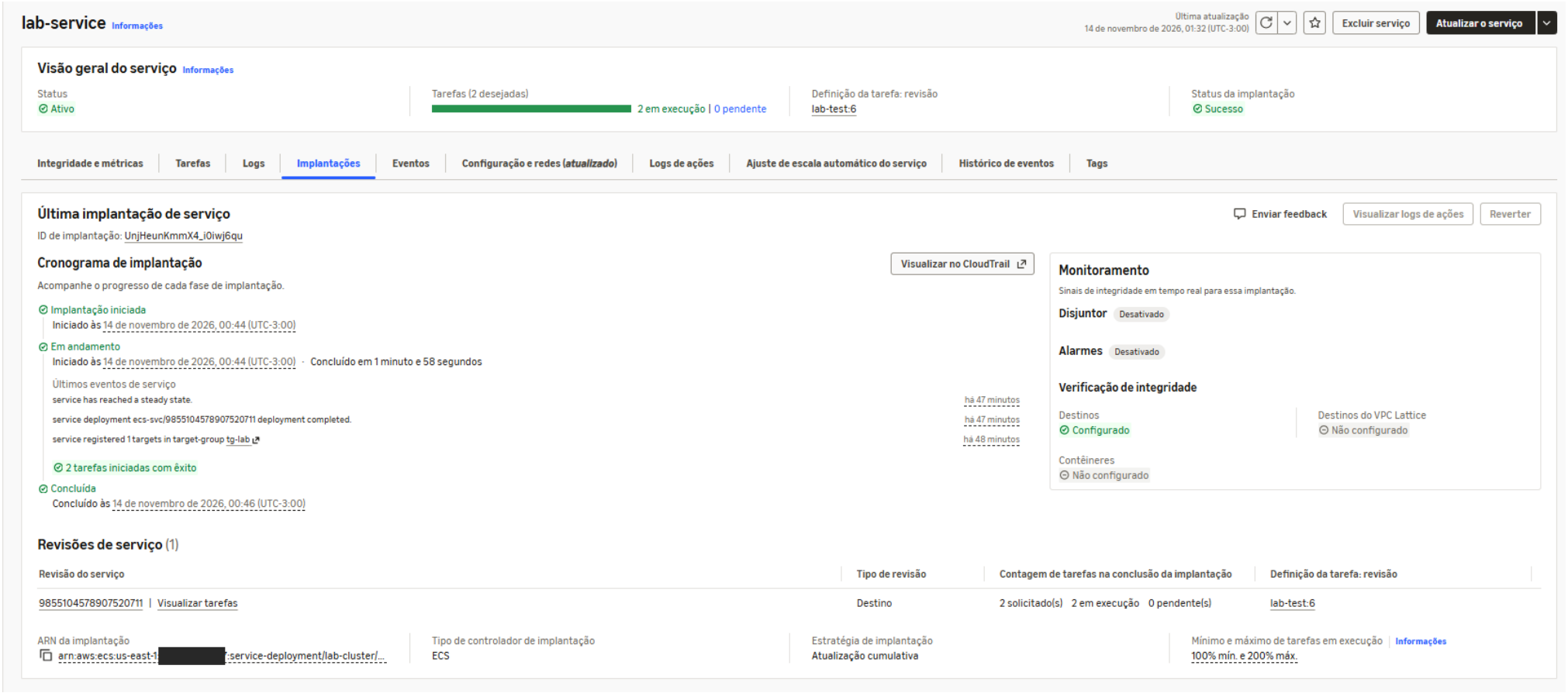
E02-a — Task definition `lab-test:6`: Fargate, Linux/X86_64, `awsvpc` e imagem `...labarquiteturanuvem:app-2.2.0`.



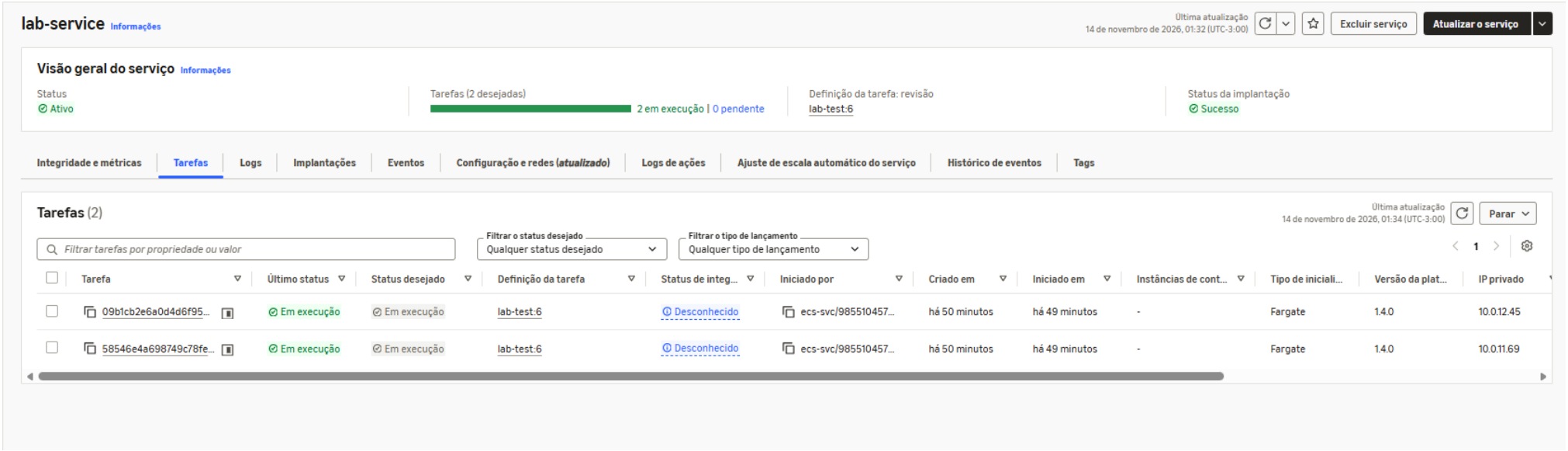
E02-b — As 15 variáveis de ambiente: as 9 `0TEL_*` e as `DB_*` com os segredos.

Evidência 3 — Service com 2 tasks na nova revisão

O `lab-service` concluiu a atualização cumulativa em 1 min 58 s, com 2 tarefas iniciadas e registradas no target group `tg-lab`. É o laço de reconciliação: as tasks novas passaram no health check antes que as antigas saíssem. As duas rodam em `lab-test:6`, uma em cada AZ (`10.0.12.45` e `10.0.11.69`).



E03-a — Aba Implantações: 2 solicitadas, 2 em execução, revisão `lab-test:6`.



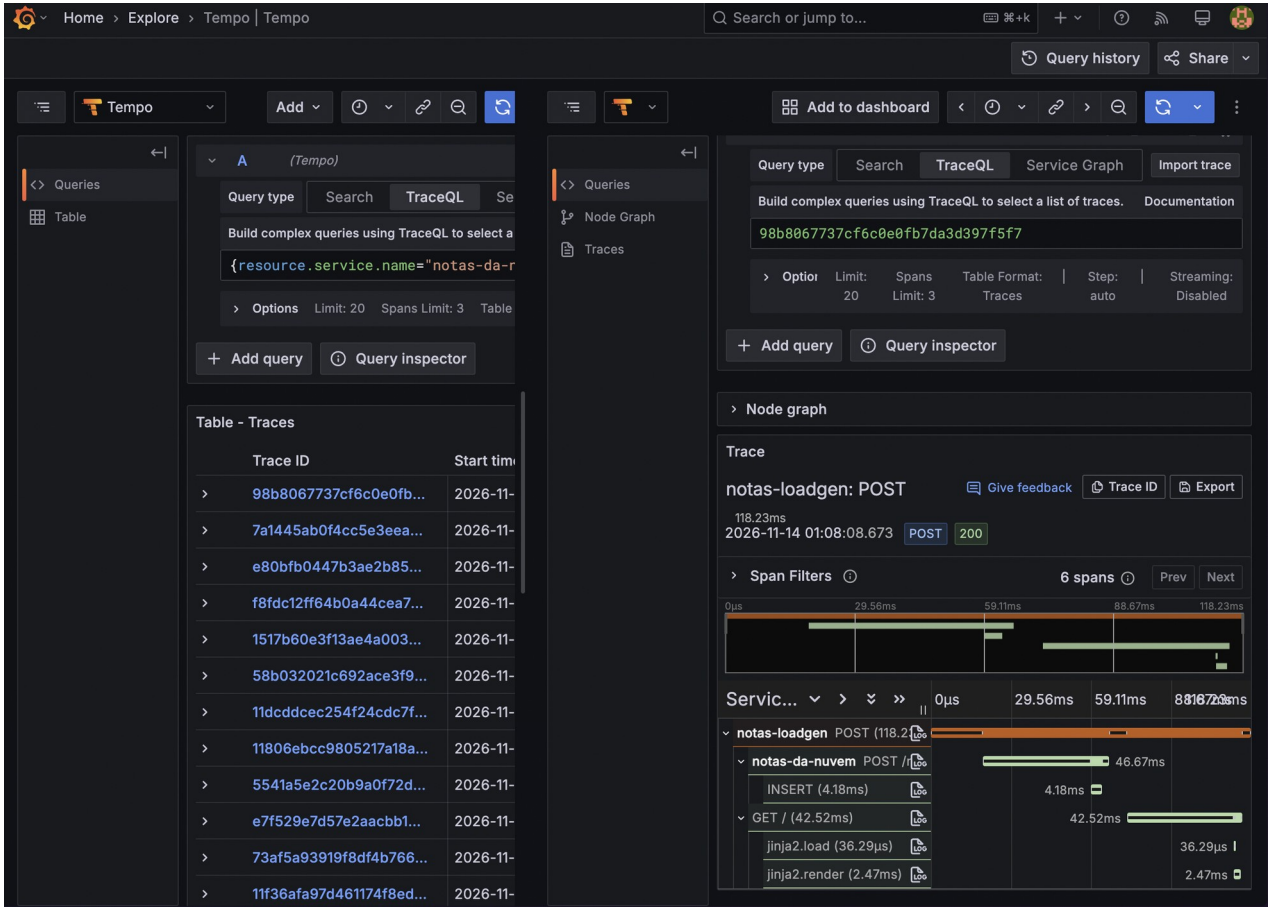
E03-b — Aba Tarefas: as 2 tasks em execução na mesma revisão.

Evidência 4 — Trace de POST /notas até o INSERT no Aurora

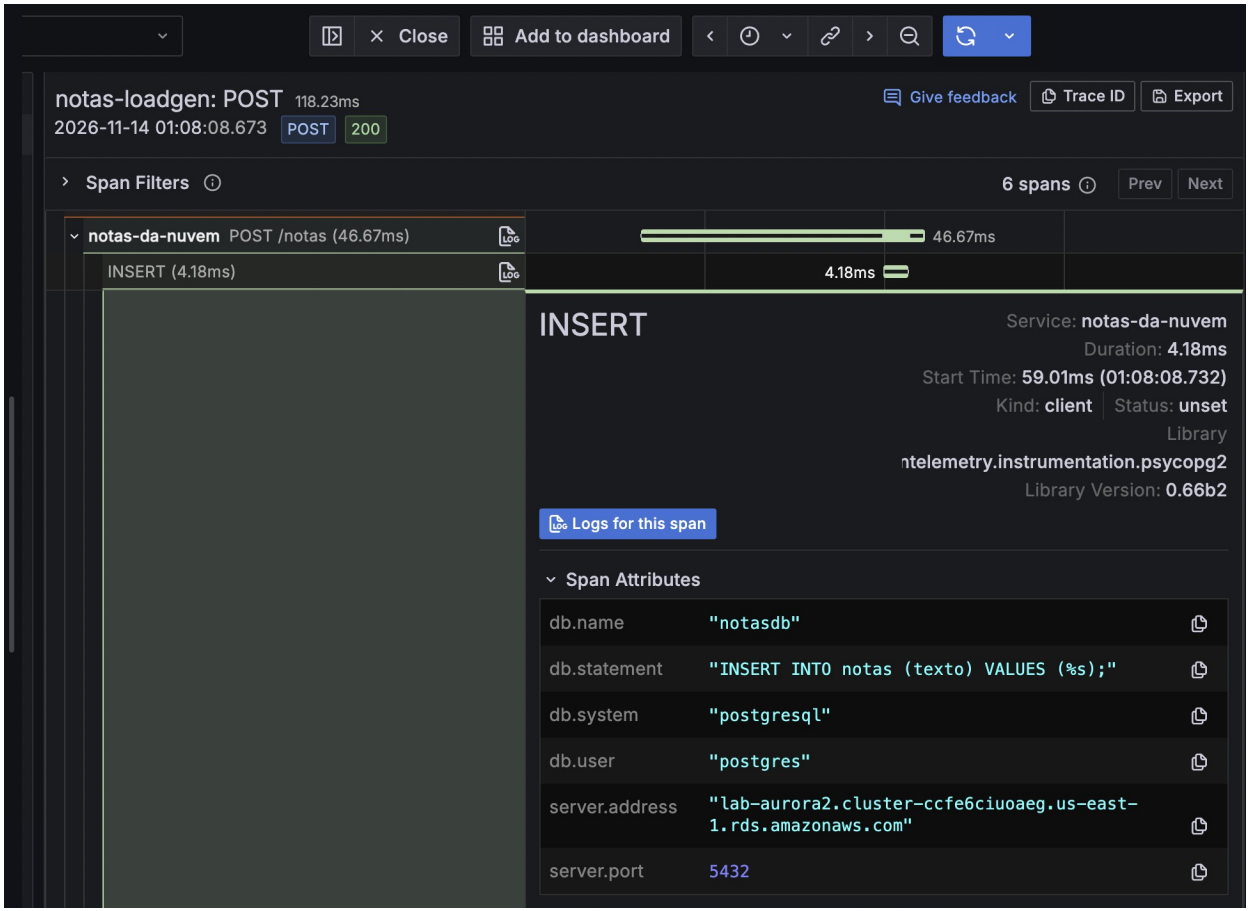
O trace começa no `notas-loadgen` e continua no `notas-da-nuvem`, porque o contexto do trace é propagado entre os dois serviços. Dentro de `POST /notas` está o span `INSERT` (4,18 ms) da instrumentação do psycopg2.

Respondendo à dica do professor: o atributo `db.statement` registra `INSERT INTO notas (texto) VALUES (%s);`. A consulta é parametrizada, e o texto da nota não aparece na telemetria.

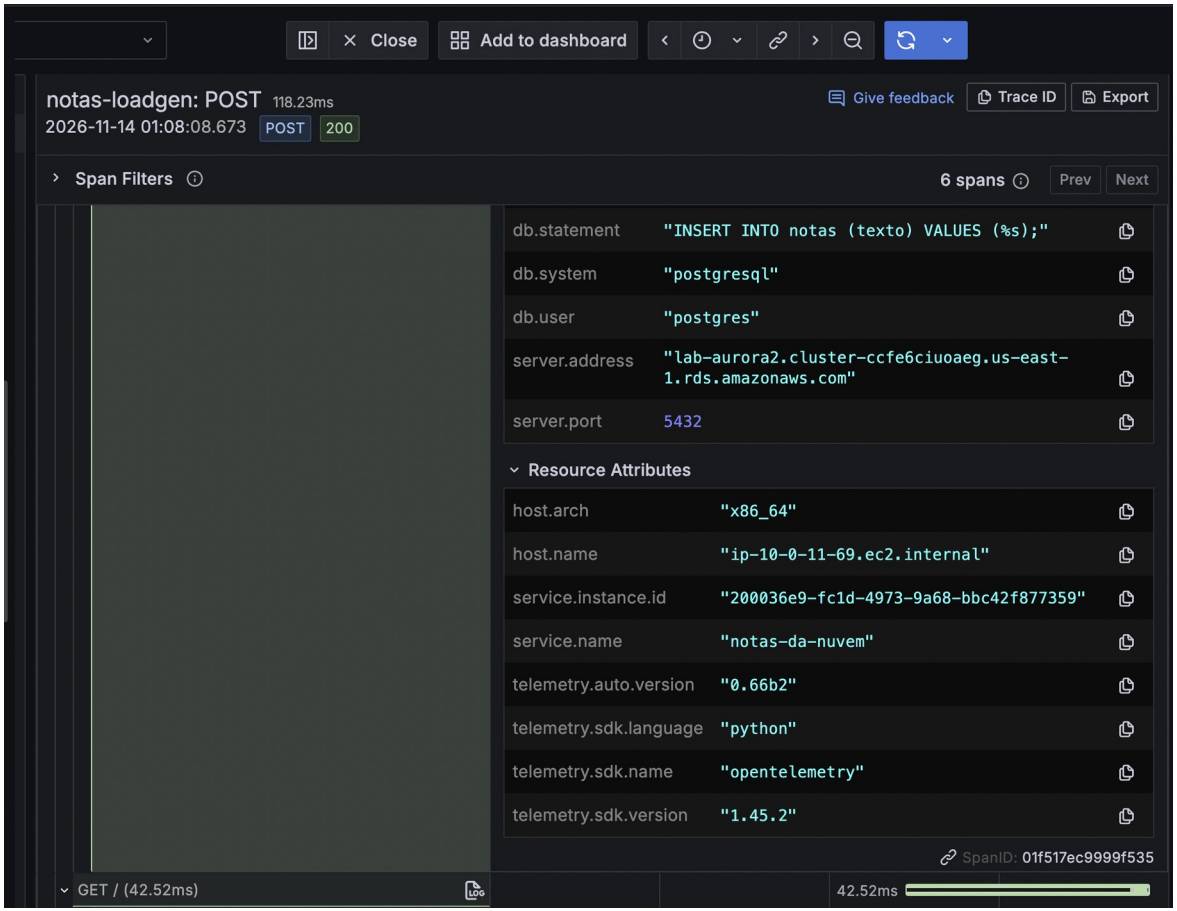
As duas tasks aparecem como instâncias diferentes, com `host.name` `ip-10-0-11-69` e `ip-10-0-12-45` e um `service.instance.id` diferente cada.



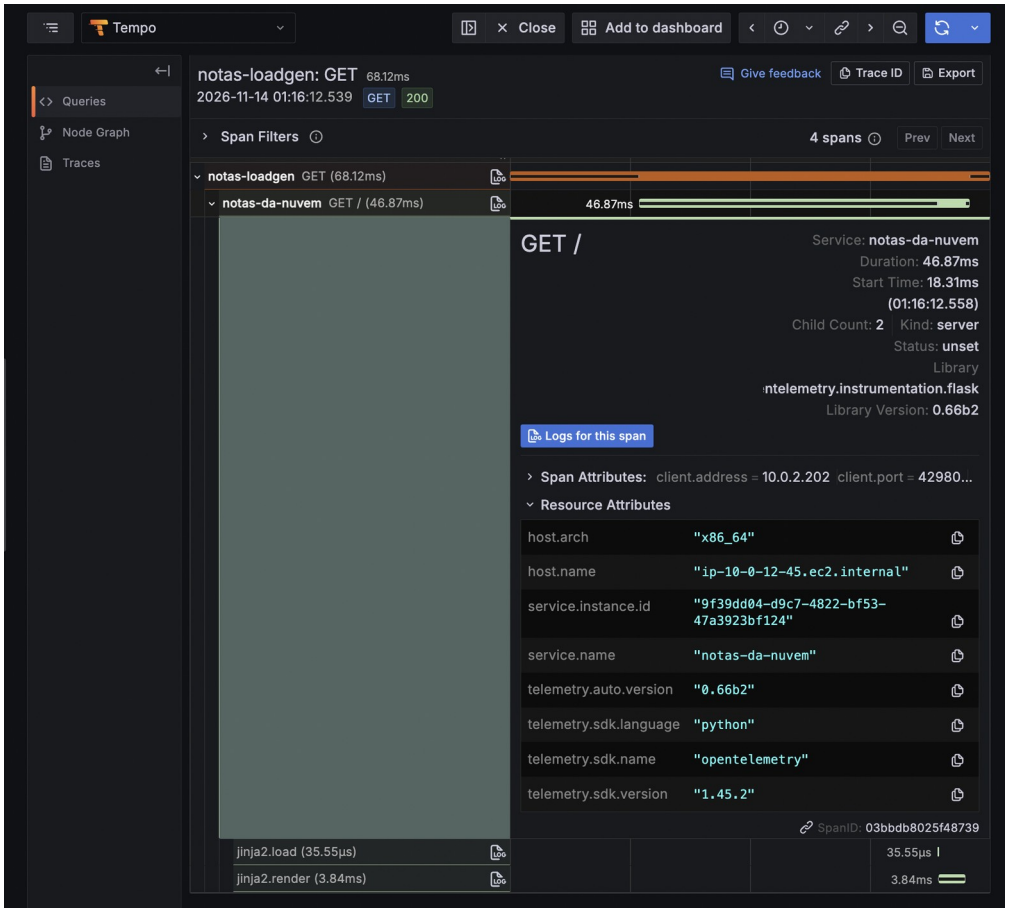
E04-a — Busca TraceQL e o trace aberto: `notas-loadgen POST` → `notas-da-nuvem POST /notas` → `INSERT`.



E04-b — Span `INSERT`: `db.statement` com parâmetro `%s`, `db.system = postgresql` e endpoint do Aurora.

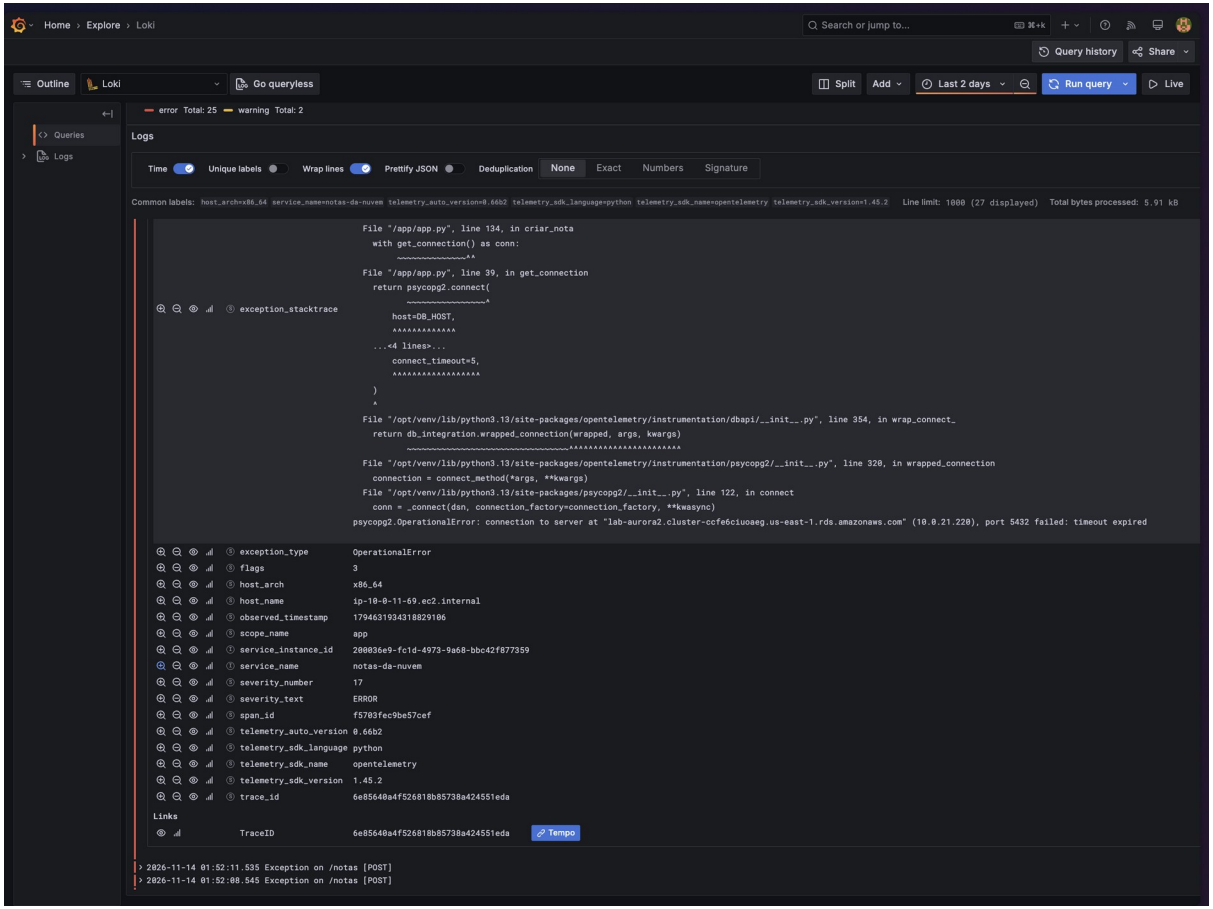


E04-c — Resource attributes da primeira task: `host.name = ip-10-0-11-69.ec2.internal`.



E04-d — Outro trace, servido pela segunda task: `host.name = ip-10-0-12-45.ec2.internal`.

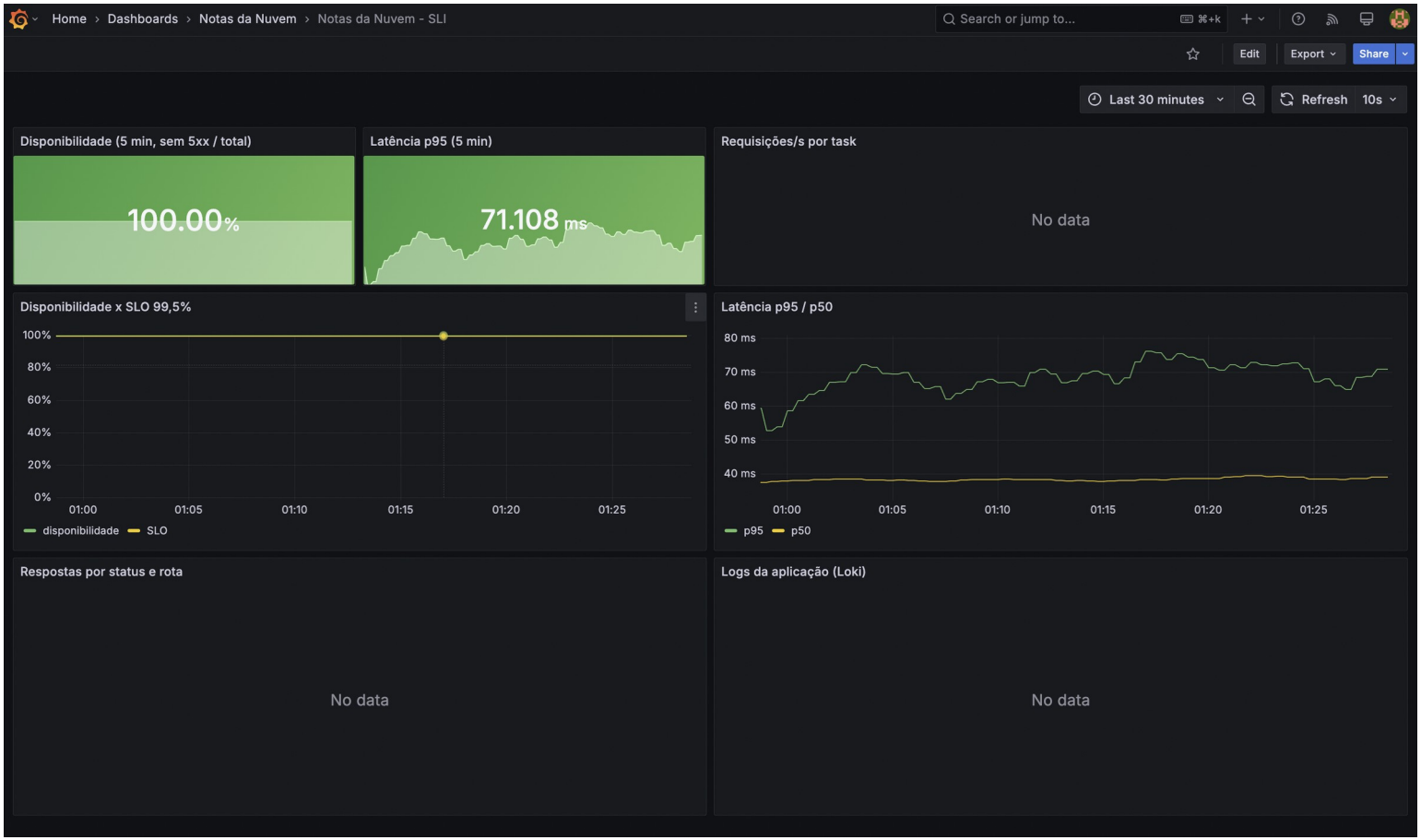
Os três pilares se correlacionam pelo trace ID. O log de erro no Loki carrega o `trace_id`, e o link **Tempo** abre o trace correspondente. O log foi capturado durante a falha da evidência 6, porque a aplicação só escreve log ao iniciar e quando há exceção.



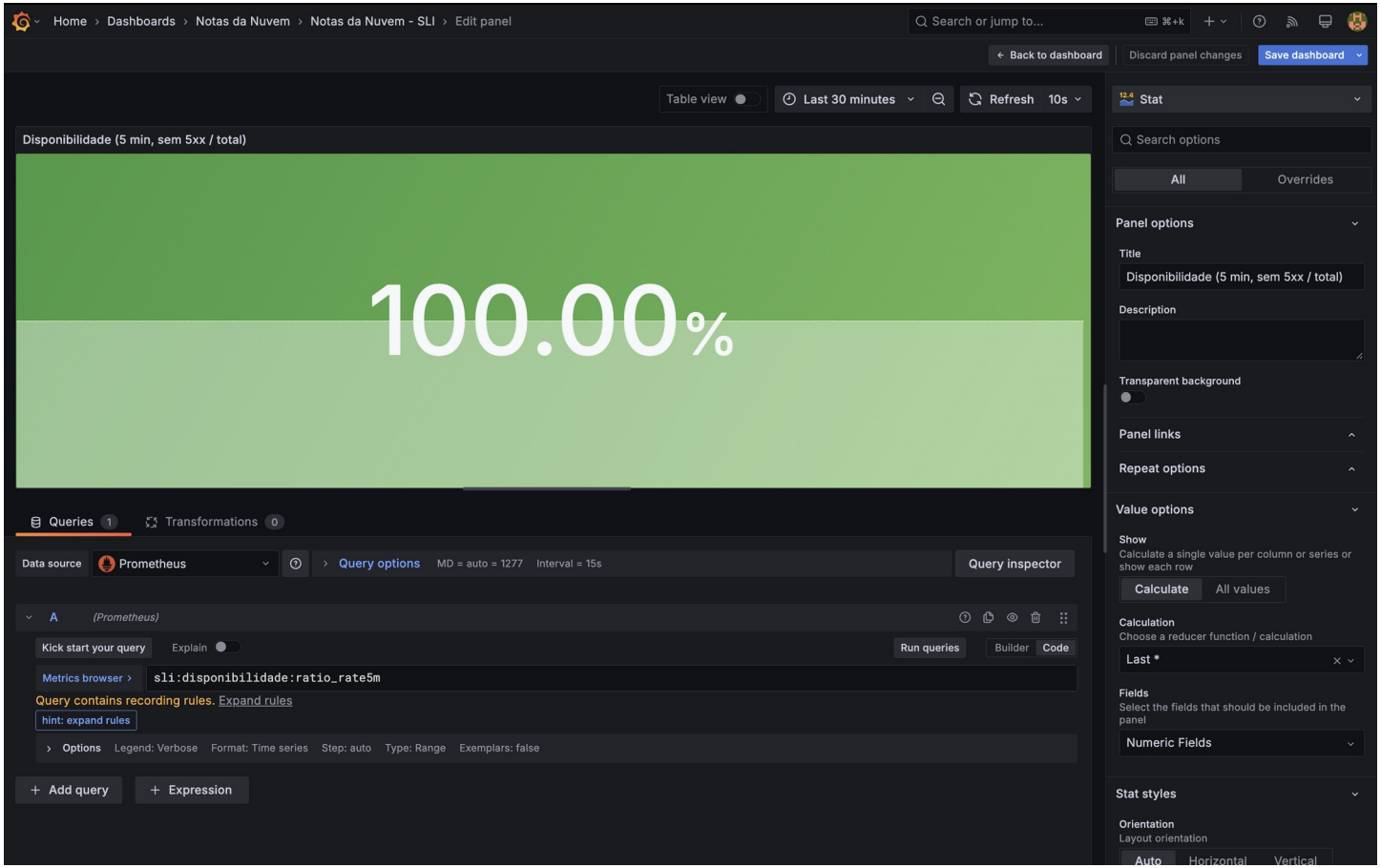
E04-e — Log no Loki com `psycopg2.OperationalError`, `trace_id` e o link para o Tempo.

Evidência 5 — Painel do SLI

O dashboard *Notas da Nuvem - SLI* mostra a disponibilidade (requisições sem 5xx sobre o total, em 5 min) e a latência p95 em 5 min. Em operação normal ficou em 100% e 71 ms. Os painéis consultam as recording rules `sli:disponibilidade:ratio_rate5m` e `sli:latencia_p95:seconds_rate5m`, as mesmas que os alertas usam. Assim, o painel e o alerta não divergem.



E05-a — Dashboard em operação normal: 100,00% de disponibilidade e p95 de 71,1 ms, com a linha do SLO de 99,5%.

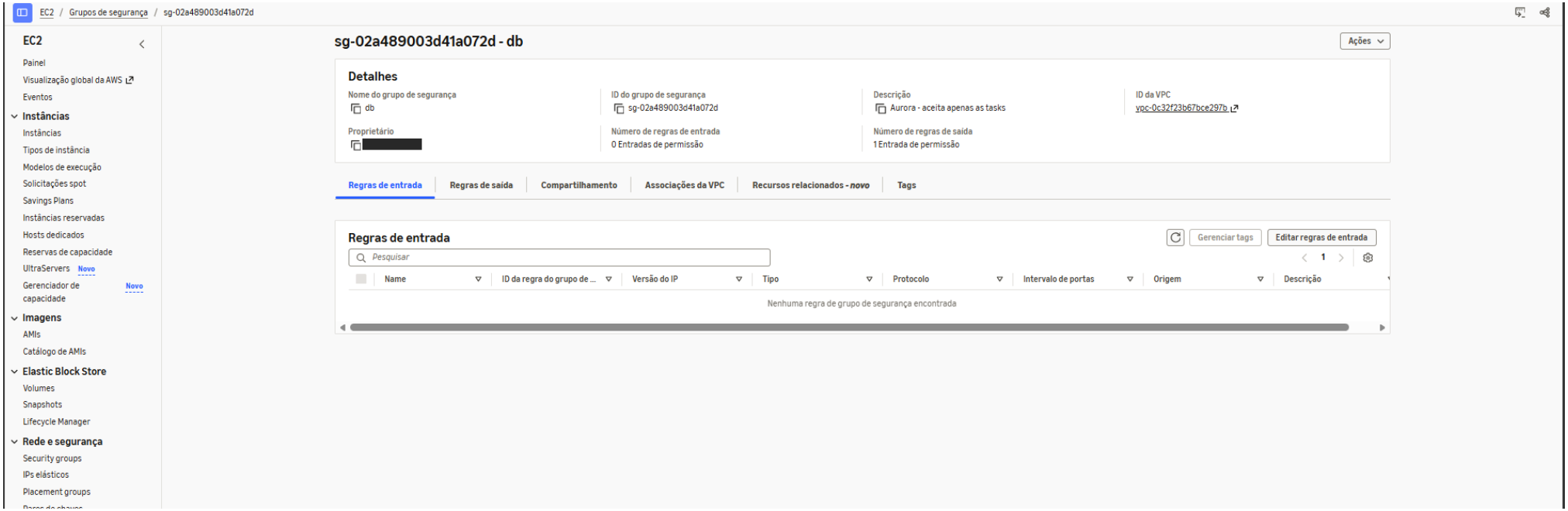


E05-b — Edição do painel de disponibilidade: a consulta `sli:disponibilidade:ratio_rate5m`.

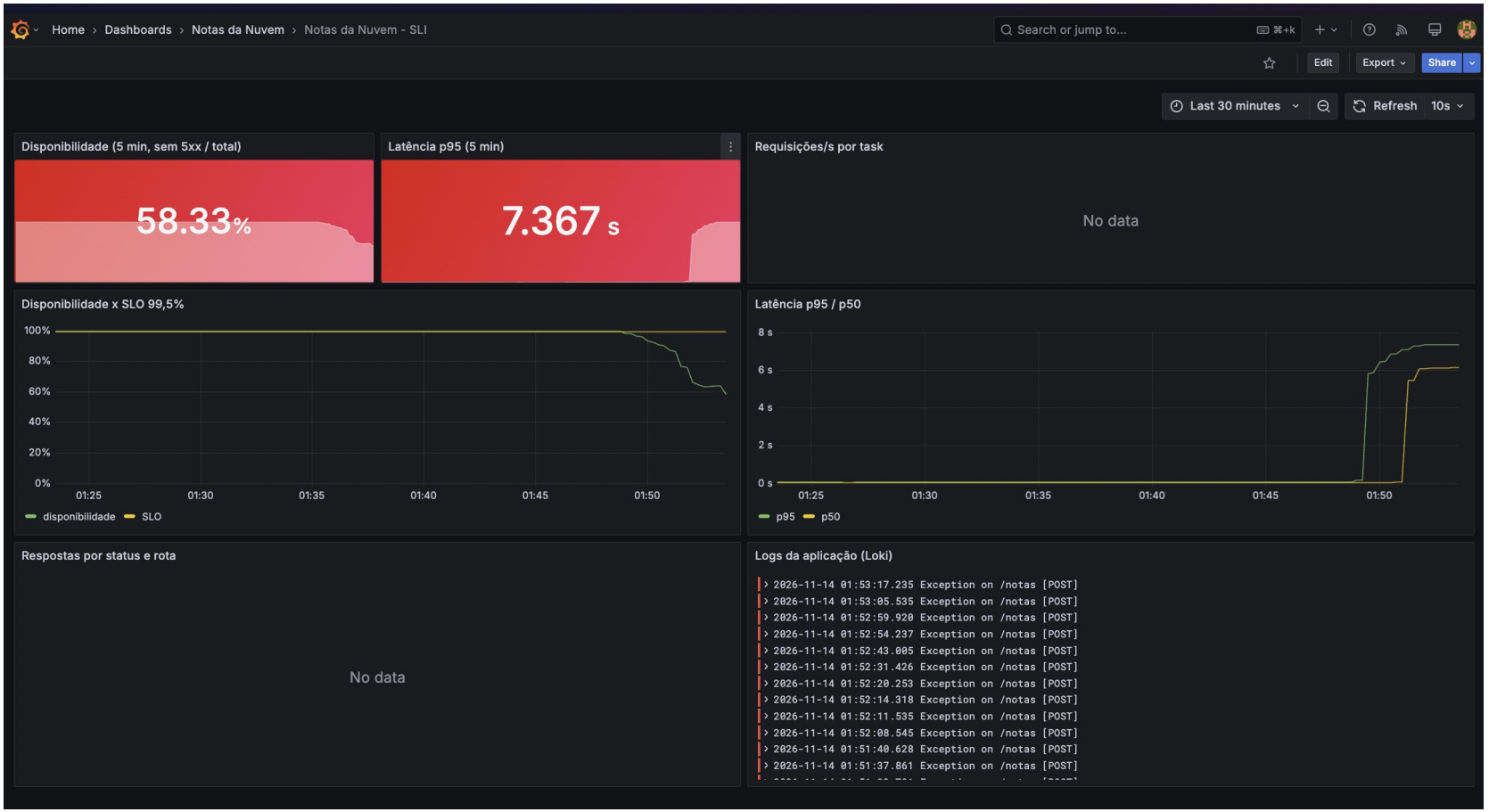
Evidência 6 — SLI caindo e trace com erro

A falha foi provocada com `terraform apply -var simulate_db_failure=true`, que remove a regra 5432 do sg `db`. O SLI caiu até 58,33%, e o p95 subiu para 7,37 s: cada tentativa espera o `connect_timeout` de 5 s. O span `POST /notas` ficou com status `error` e a mensagem `OperationalError: ... port 5432 failed: timeout expired`.

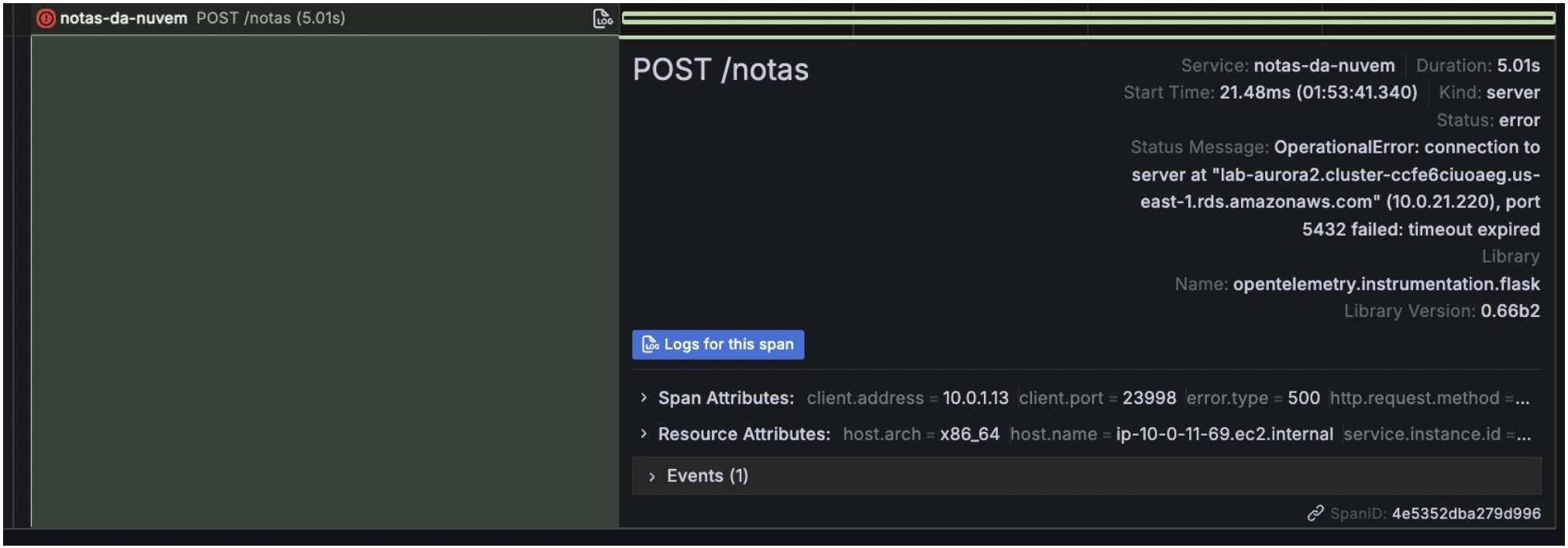
O ALB continuou considerando as duas tasks saudáveis durante a falha, porque `/health` não consulta o banco. O balanceador viu as requisições, mas quem explicou a falha foi a aplicação, como discutido na dinâmica da Seção 7. Depois de recolocar a regra, a disponibilidade voltou a 99,69% e o p95 a 99,7 ms.



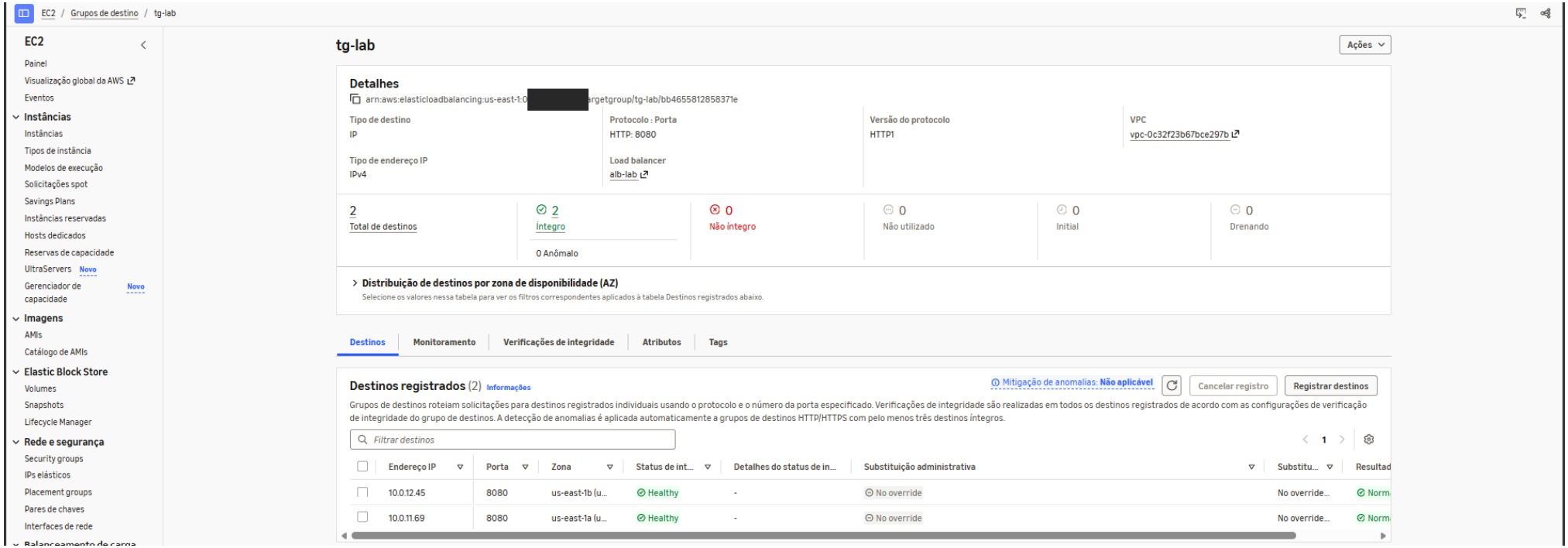
E06-a — sg `db` sem nenhuma regra de entrada durante a falha.



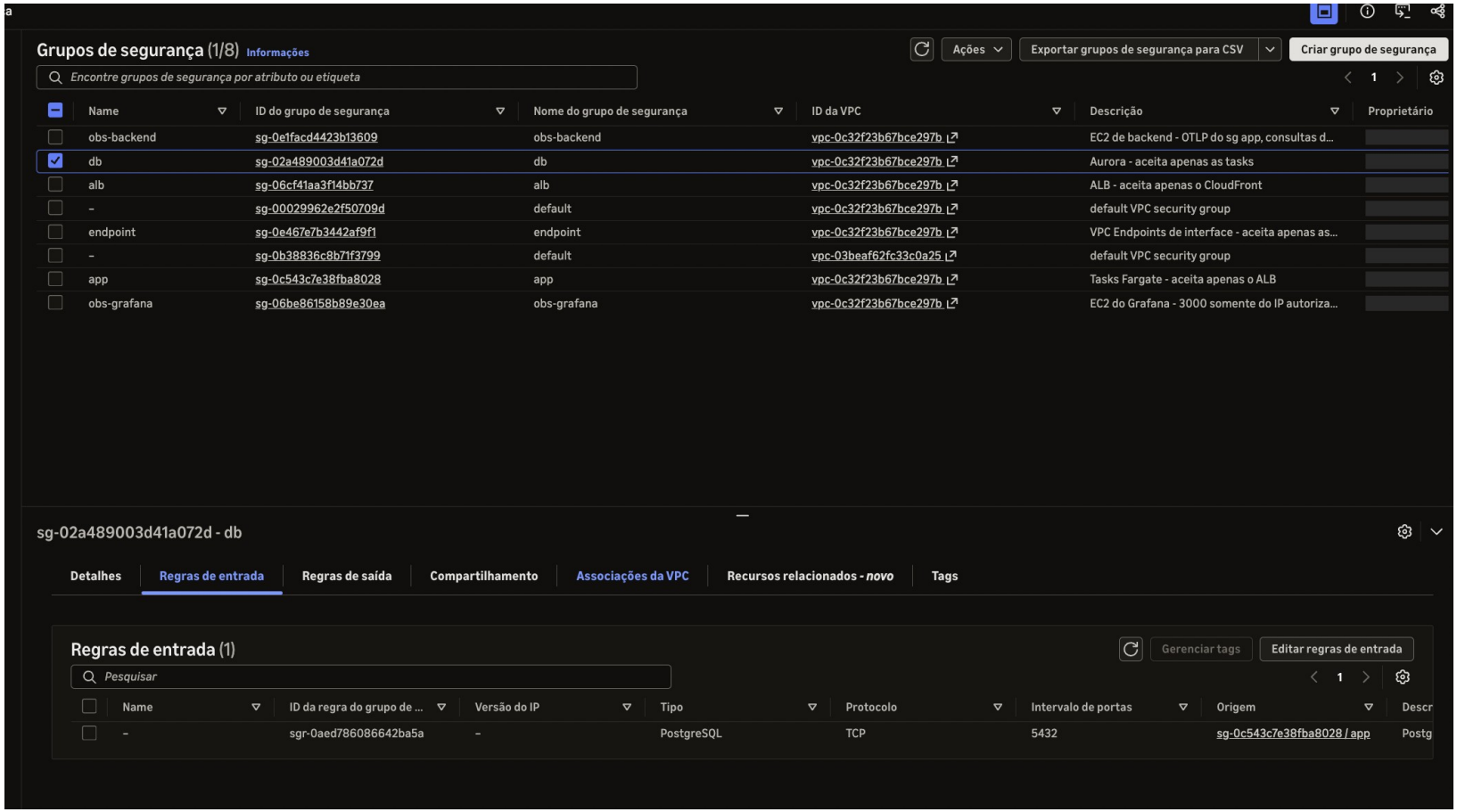
E06-b — Dashboard durante a falha: disponibilidade em 58,33%, p95 em 7,367 s e logs `Exception on /notas [POST]`.



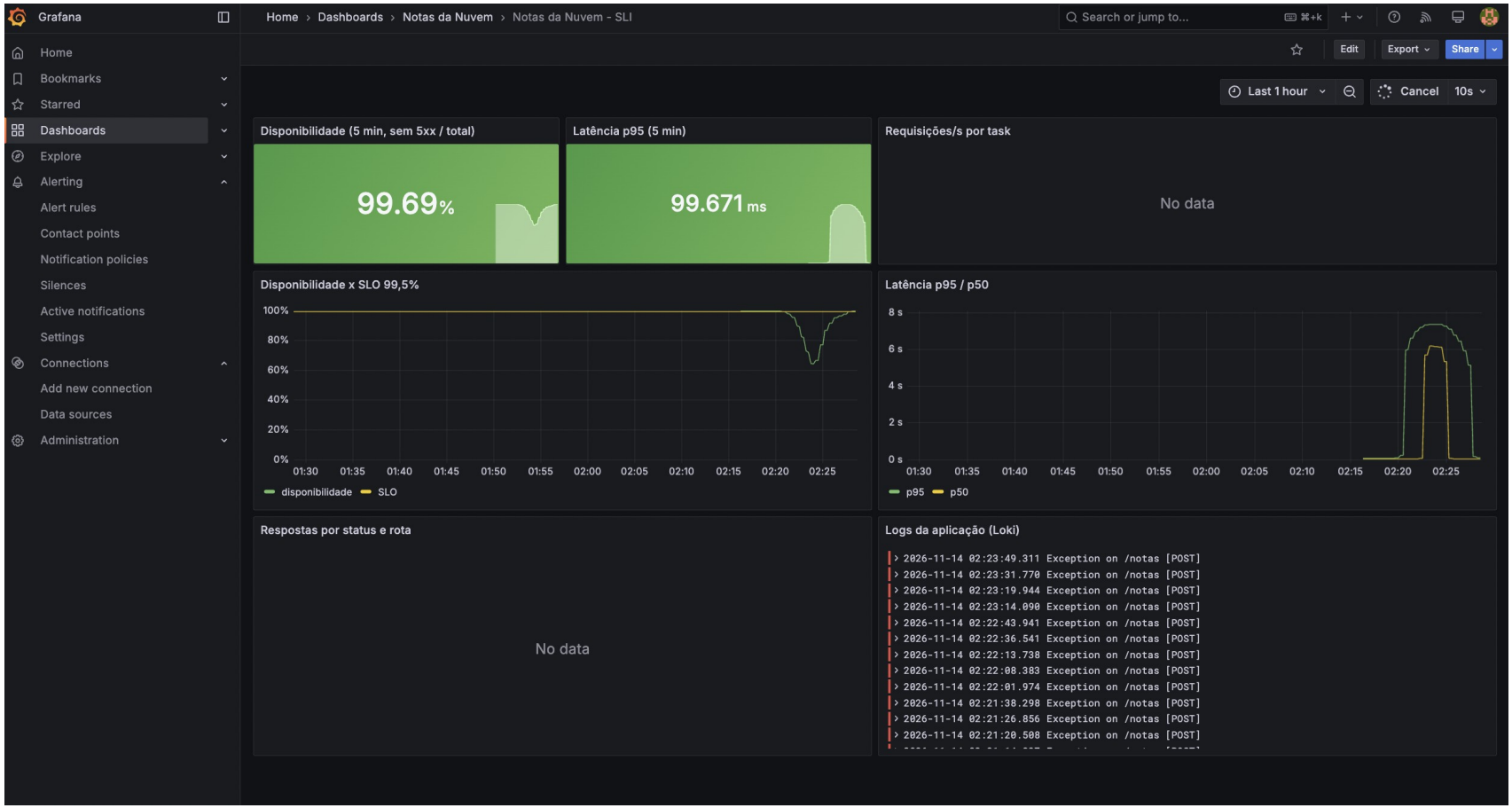
E06-c — Span `POST /notas` com status error e timeout de conexão ao Aurora na porta 5432.



E06-e — Target group `tg-lab` durante a falha: 2 destinos Healthy.



E06-f — sg db com a regra PostgreSQL 5432 de origem sg app restaurada.



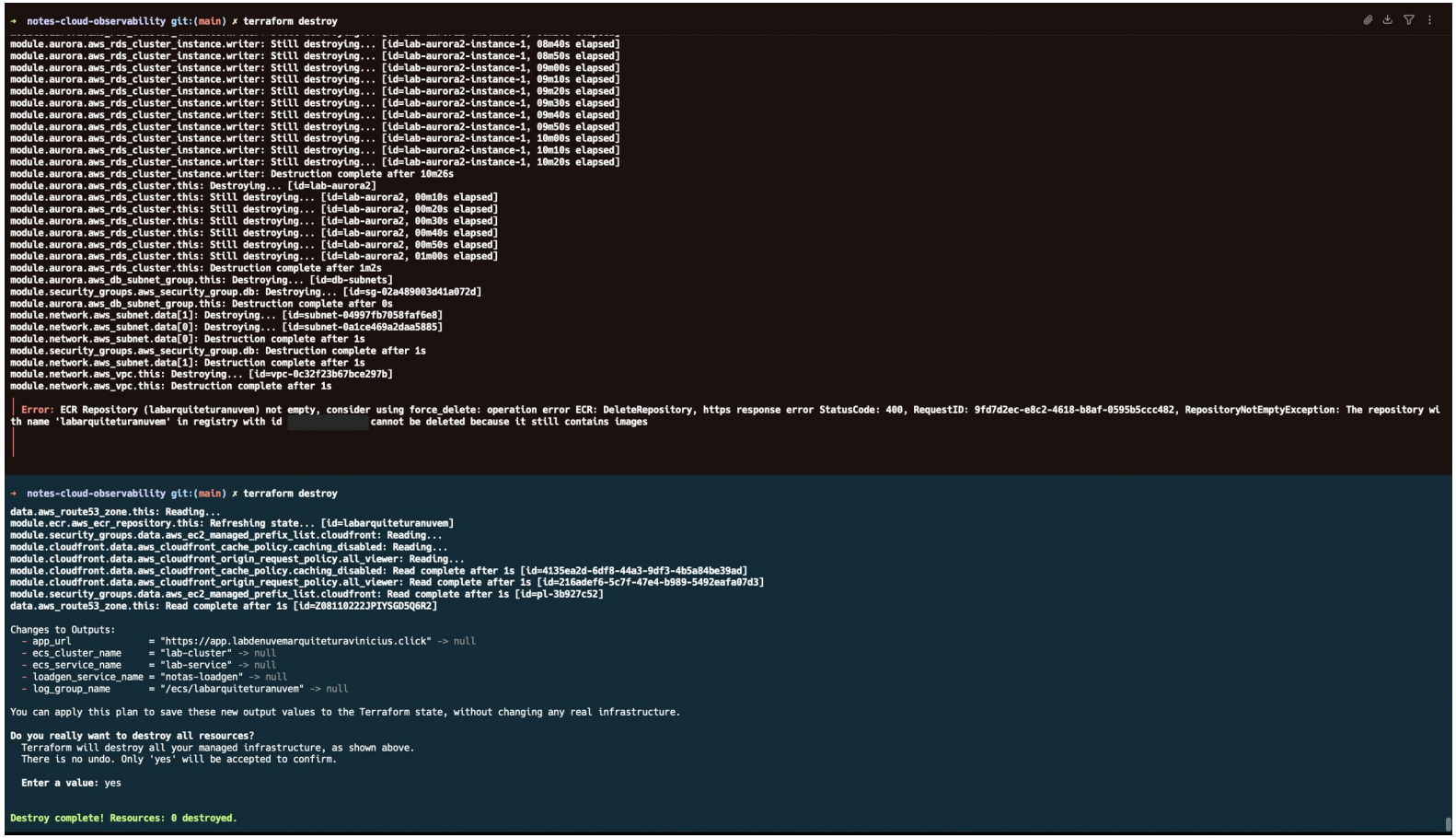
E06-g — Última hora: a queda, a recuperação e a disponibilidade de volta a 99,69%.

Evidência 7 — Destruição do ambiente (terraform destroy)

O ambiente foi destruído por completo com `terraform destroy`. O plano listou 99 recursos, e 98 foram removidos na primeira execução, entre eles as duas EC2 de observabilidade (`lab-obs-grafana` em 52 s e `lab-obs-backend` em 42 s), o Elastic IP, o Aurora (writer em 10 min 26 s), o ALB, o CloudFront, os VPC Endpoints, as subnets e, por último, a VPC.

O único recurso que falhou foi o repositório ECR da aplicação, `labarquiteturanuvem`: a AWS recusa apagar um repositório que ainda tem imagens (`RepositoryNotEmptyException`). Os 5 repositórios espelhados da stack (`lab-obs/*`) foram apagados normalmente, porque têm `force_delete = true`; o da aplicação, criado no laboratório da Aula 10, não tinha.

O grupo apagou o repositório manualmente e rodou o `terraform destroy` de novo. A segunda execução atualizou o estado, não encontrou mais nada para remover e terminou com `Destroy complete! Resources: 0 destroyed.`, o que confirma que nenhum recurso gerenciado ficou na conta.



E07-a — Fim da primeira execução (VPC destruída e o erro do ECR) e a segunda execução terminando com `Destroy complete! Resources: 0 destroyed.`

Trechos do log da primeira e da segunda execução (Account ID substituído por `<ACCOUNT_ID>`):

```
$ terraform destroy
Plan: 0 to add, 0 to change, 99 to destroy.
...
module.observability[0].aws_eip.grafana: Destruction complete after 2s
module.observability[0].aws_instance.grafana: Destruction complete after 52s
module.observability[0].aws_instance.backend: Destruction complete after 42s
...
module.aurora.aws_rds_cluster_instance.writer: Destruction complete after 10m26s
module.aurora.aws_rds_cluster.this: Destruction complete after 1m2s
...
module.network.aws_vpc.this: Destruction complete after 1s

Error: ECR Repository (labarquiteturanuvem) not empty, consider using force_delete:
  RepositoryNotEmptyException: The repository with name 'labarquiteturanuvem' in registry
  with id '<ACCOUNT_ID>' cannot be deleted because it still contains images

# repositório labarquiteturanuvem apagado manualmente

$ terraform destroy
module.ecr.aws_ecr_repository.this: Refreshing state... [id=labarquiteturanuvem]
...
Destroy complete! Resources: 0 destroyed.
```

O log completo da primeira execução (4.354 linhas) acompanha esta entrega como arquivo separado, terraform-destroy.log, com o Account ID e os IPs públicos substituídos.

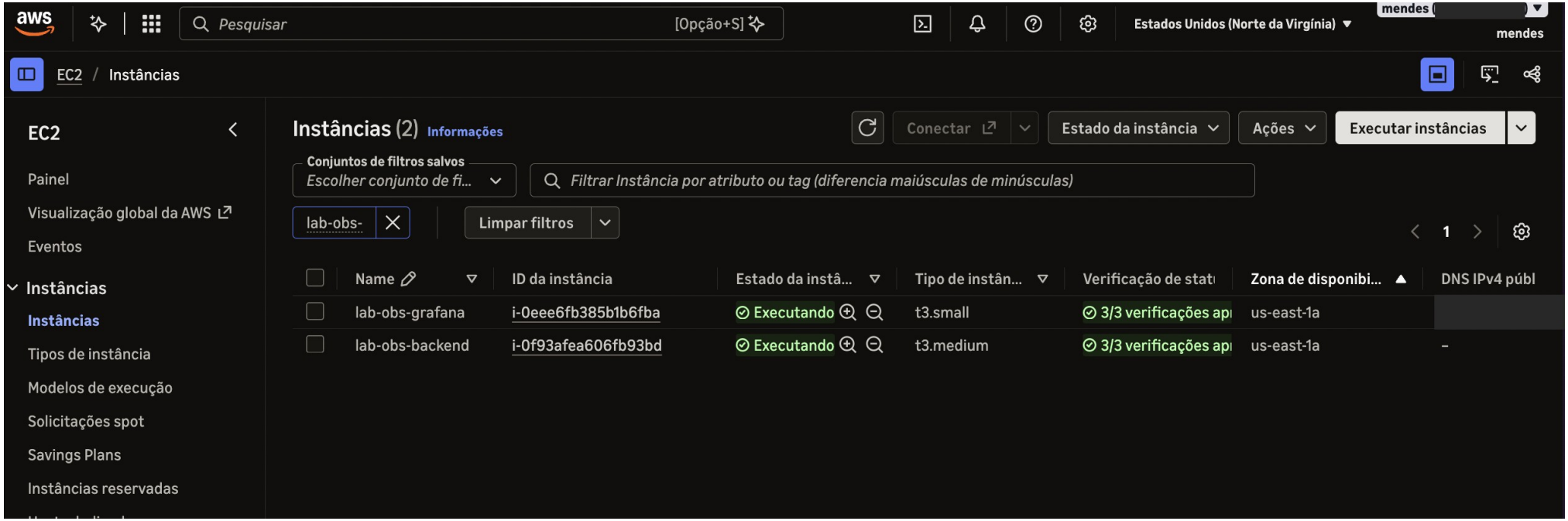
Lição aprendida: declarar `force_delete = true` também no repositório da aplicação, para que o `terraform destroy` termine sem passo manual.

Evidências dos extras (8 a 13)

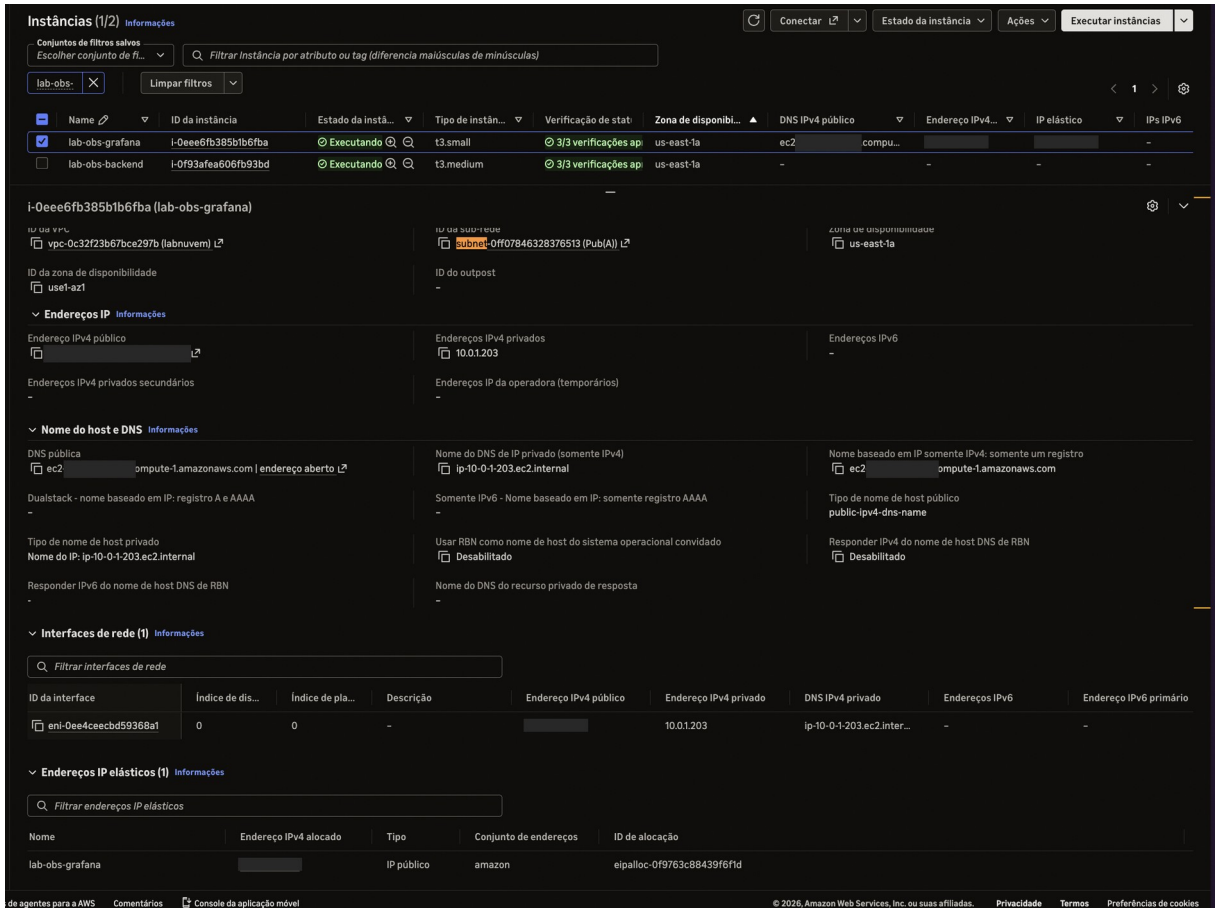
Evidência 8 — Grafana na subnet pública, backend na privada

A stack foi separada em duas EC2 na VPC `labnuvem`, as duas em us-east-1a. A `lab-obs-grafana` (t3.small) fica na subnet `Pub(A)`, com Elastic IP. A `lab-obs-backend` (t3.medium) fica na `PrivateFargate(A)` (`10.0.11.0/24`), sem IP público nem DNS público.

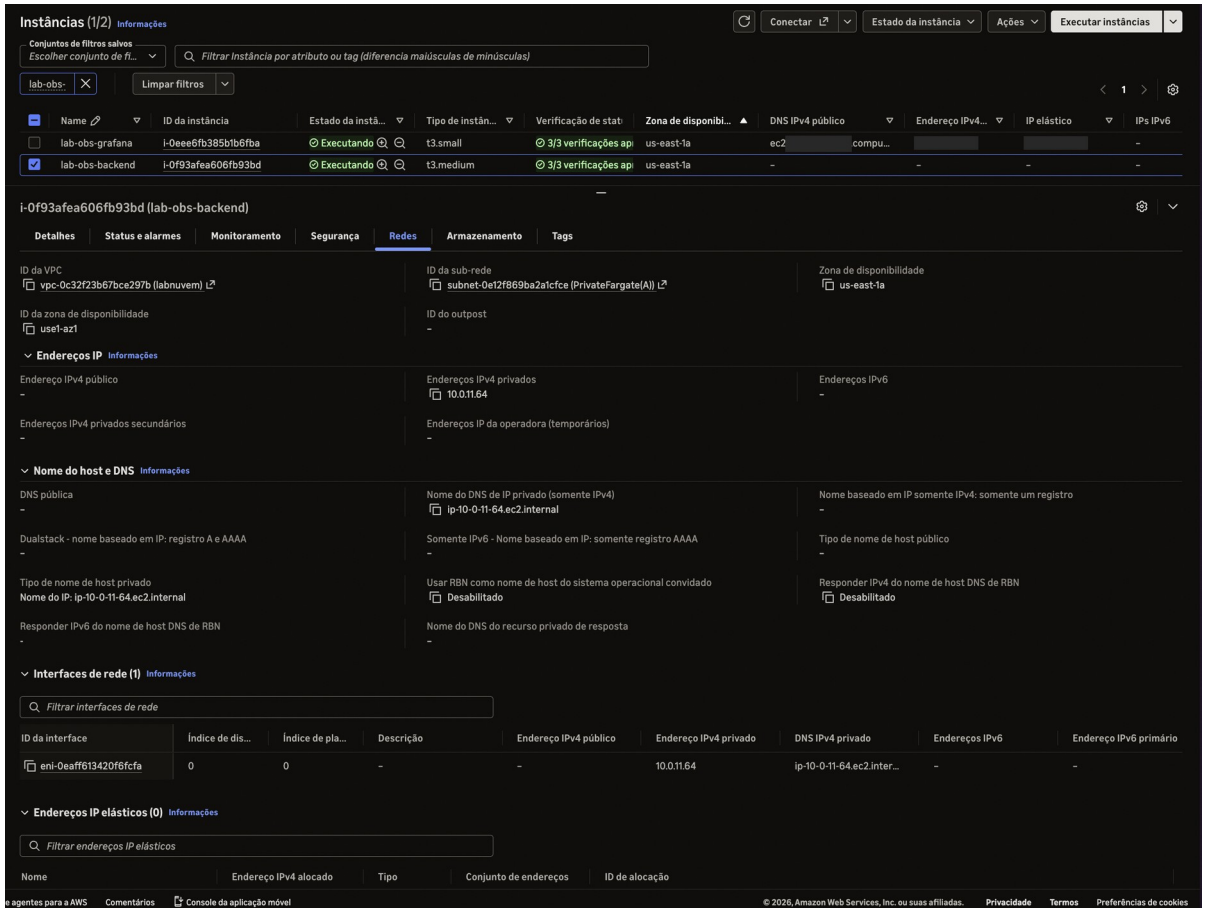
A tabela de rotas da subnet privada tem só a rota `local` e a do endpoint gateway do S3. Não há `0.0.0.0/0` para Internet Gateway nem para NAT. O backend instala pacotes pelo endpoint do S3 e baixa as imagens do ECR pelos VPC Endpoints.



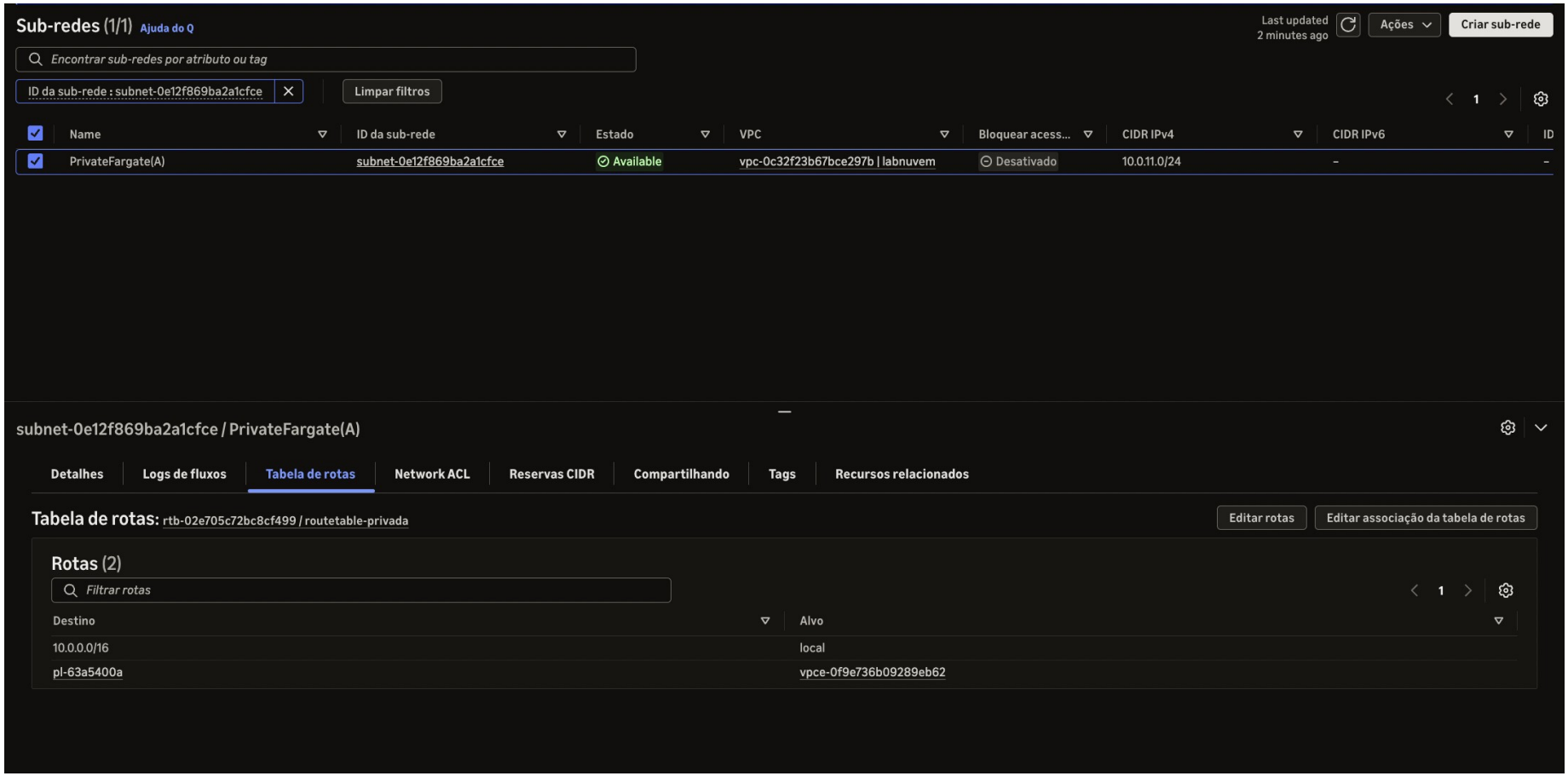
E08-a — As duas instâncias `lab-obs-*` em execução; o backend sem DNS IPv4 público.



E08-b — `lab-obs-grafana`: subnet `Pub(A)`, IPv4 público e Elastic IP (tarjados), IP privado `10.0.1.203`.



E08-c — `lab-obs-backend`: subnet `PrivateFargate(A)`, IPv4 público "–", IP privado `10.0.11.64`.

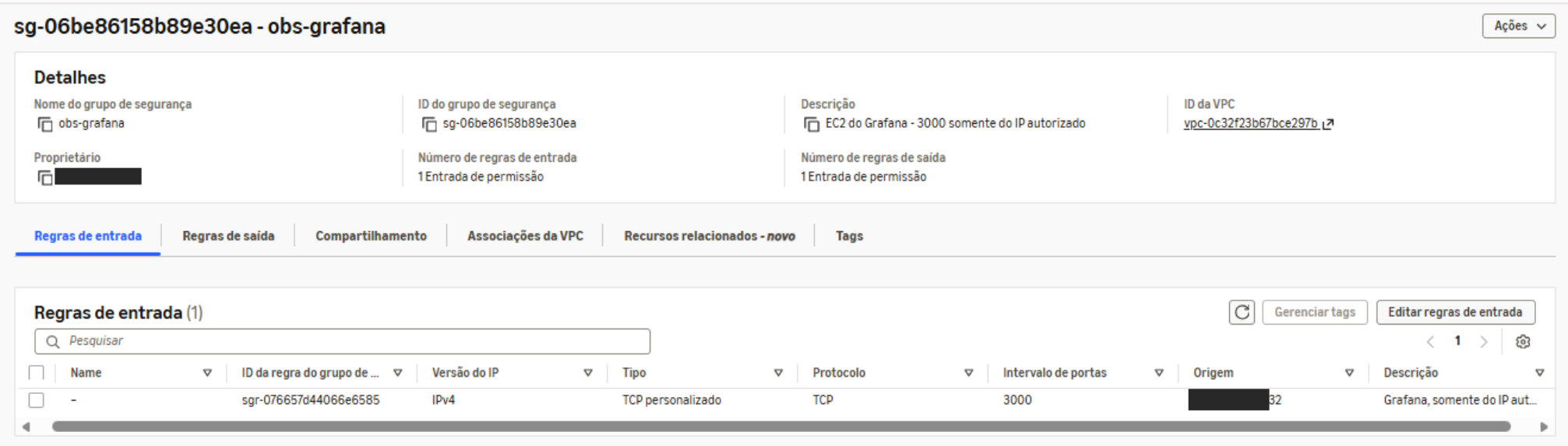


E08-d — Tabela de rotas `routetable-privada`: apenas `10.0.0.0/16 → local` e a prefix list do S3 → VPC Endpoint.

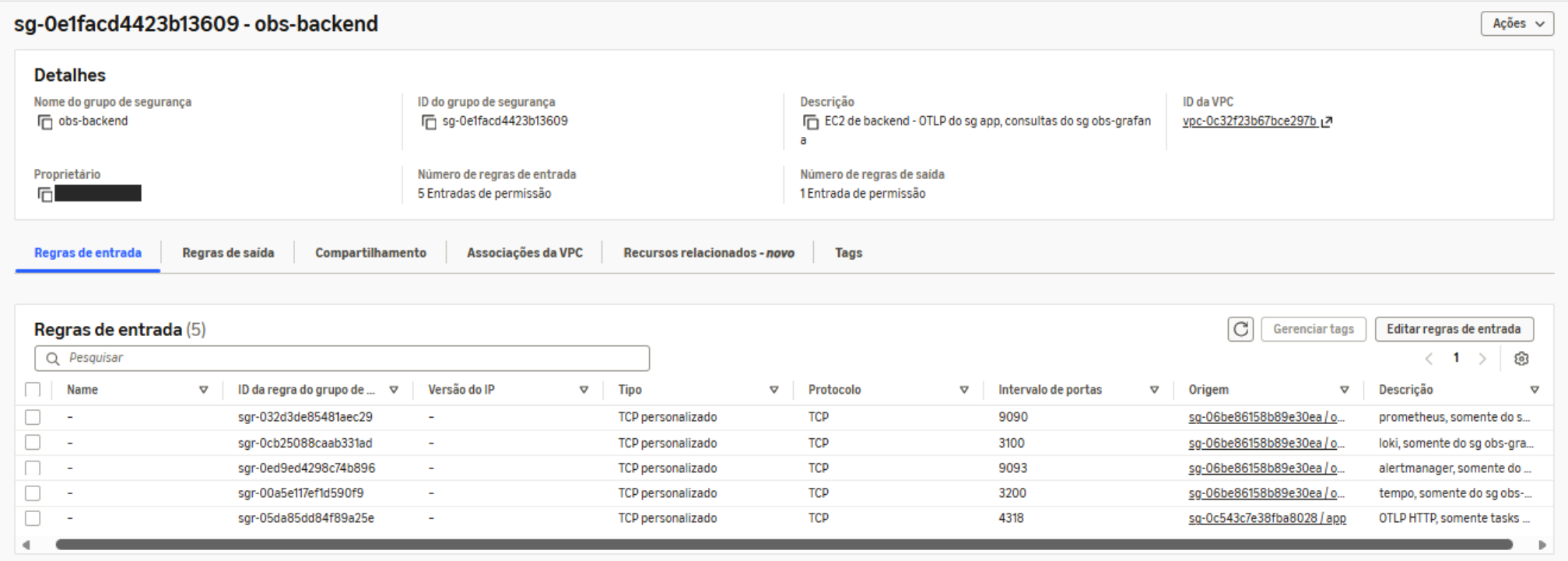
Evidência 9 — Portas restritas nos security groups

O `obs-grafana` tem uma única regra de entrada: TCP 3000 a partir de um IP `/32`. A validação do Terraform recusa `0.0.0.0/0` nessa variável.

O `obs-backend` aceita 4318 (OTLP HTTP) só do sg `app`, onde rodam as tasks da aplicação e do loadgen. As consultas, nas portas 9090 (Prometheus), 3100 (Loki), 3200 (Tempo) e 9093 (Alertmanager), só são aceitas a partir do sg `obs-grafana`. As regras referenciam grupos, e não IPs, como na Seção 5 da apostila da Aula 10.



E09-a — sg `obs-grafana`: 1 regra, TCP 3000, origem `/32` (IP tarjado).

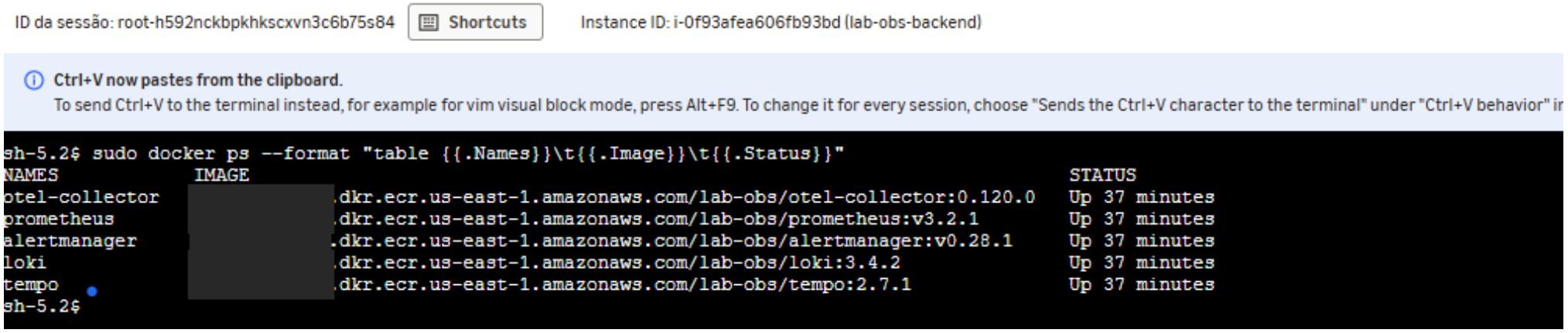


E09-b — sg `obs-backend`: 9090, 3100, 9093 e 3200 de `obs-grafana`; 4318 de `app`.

Evidência 10 — OTLP, Tempo, Loki e Prometheus na subnet privada

Na EC2 privada, acessada pelo Session Manager (sem chave SSH e sem porta 22), rodam cinco containers, um por componente. As versões estão fixadas, e as imagens foram espelhadas do Docker Hub para o ECR (`lab-obs/*`), para que o backend as baixe pelos VPC Endpoints, sem NAT Gateway.

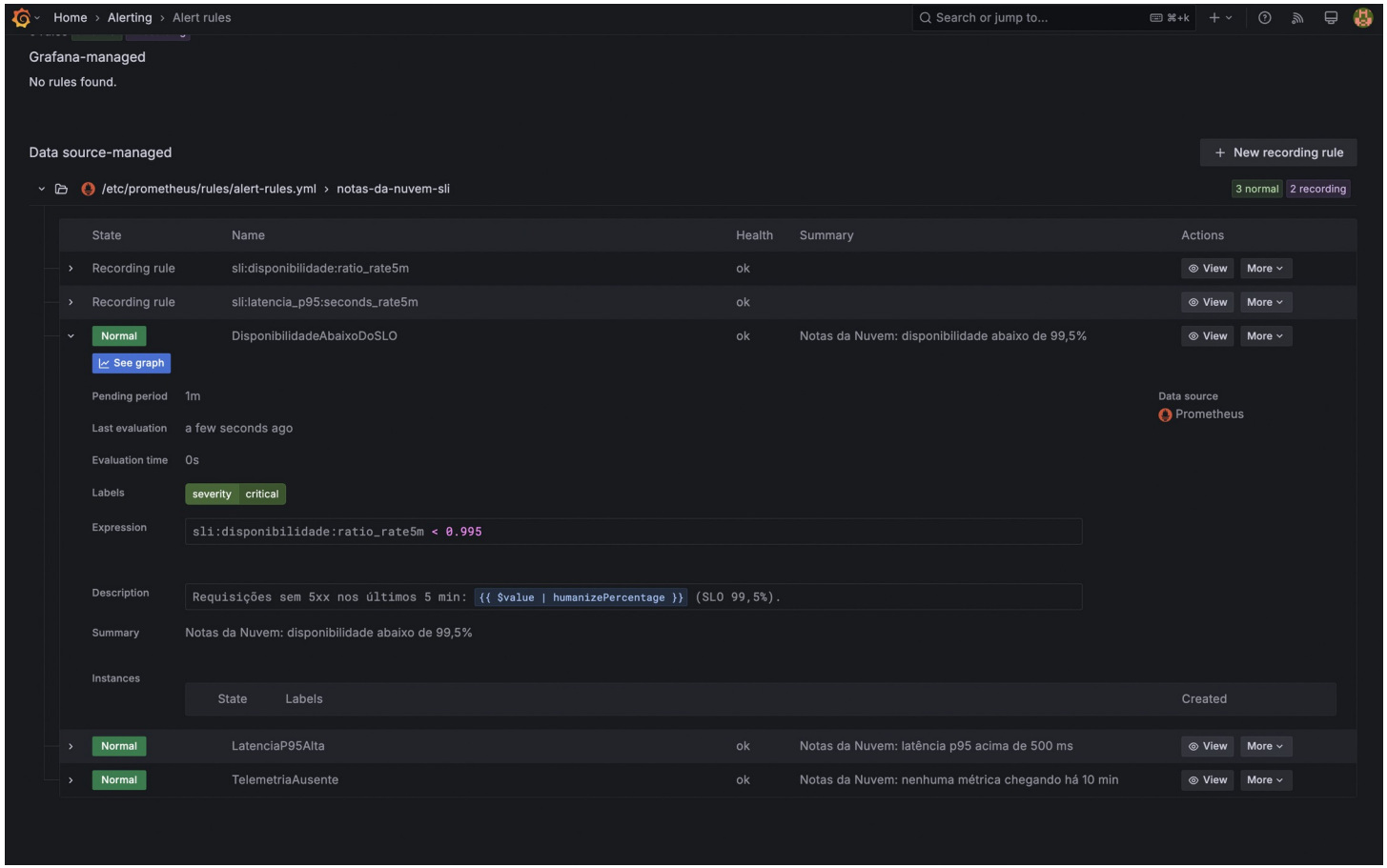
Componente	Imagem	Papel
OpenTelemetry Collector	`otel-collector:0.120.0`	Recebe OTLP na 4318 e distribui os três sinais
Prometheus	`prometheus:v3.2.1`	Métricas (remote write), recording rules e alertas
Tempo	`tempo:2.7.1`	Traces
Loki	`loki:3.4.2`	Logs (OTLP nativo)
Alertmanager	`alertmanager:v0.28.1`	Agrupa e envia os alertas ao Discord



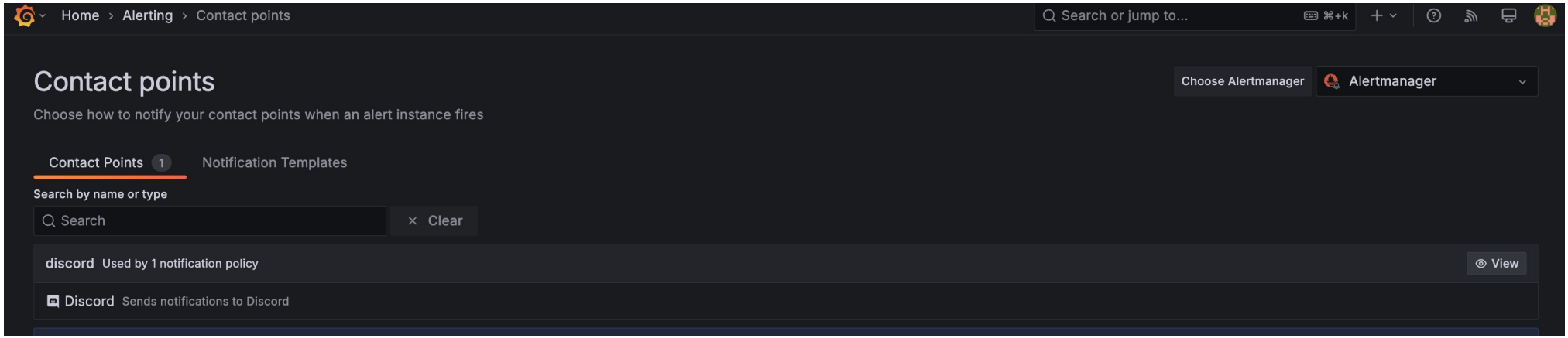
E10-a — `sudo docker ps` na `lab-obs-backend` via Session Manager: os 5 containers Up há 37 minutos.

Evidência 11 — Alertmanager integrado ao Grafana

As regras ficam no Prometheus (`/etc/prometheus/rules/alert-rules.yml`, grupo `notas-da-nuvem-sli`) e aparecem no Grafana como *Data source-managed*. No Grafana, o Alertmanager externo é data source e expõe o contact point `discord`, configurado no `alertmanager.yml`. As regras estão descritas na seção de SLI, mais abaixo.



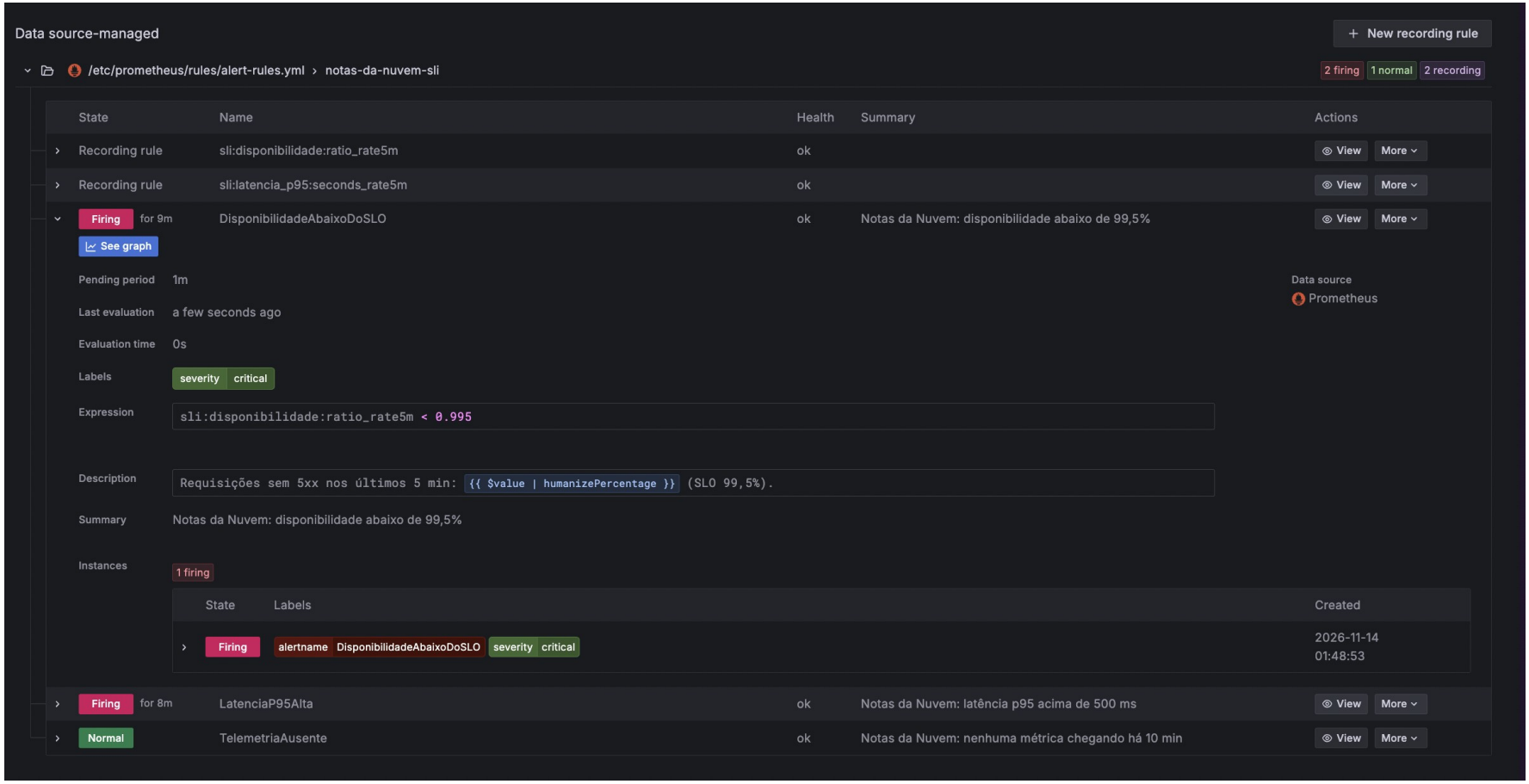
E11-a — Alert rules: 2 recording rules e 3 alertas; `DisponibilidadeAbaixoDoSLO` expandida, com expressão, pending period de 1 min e severidade critical.



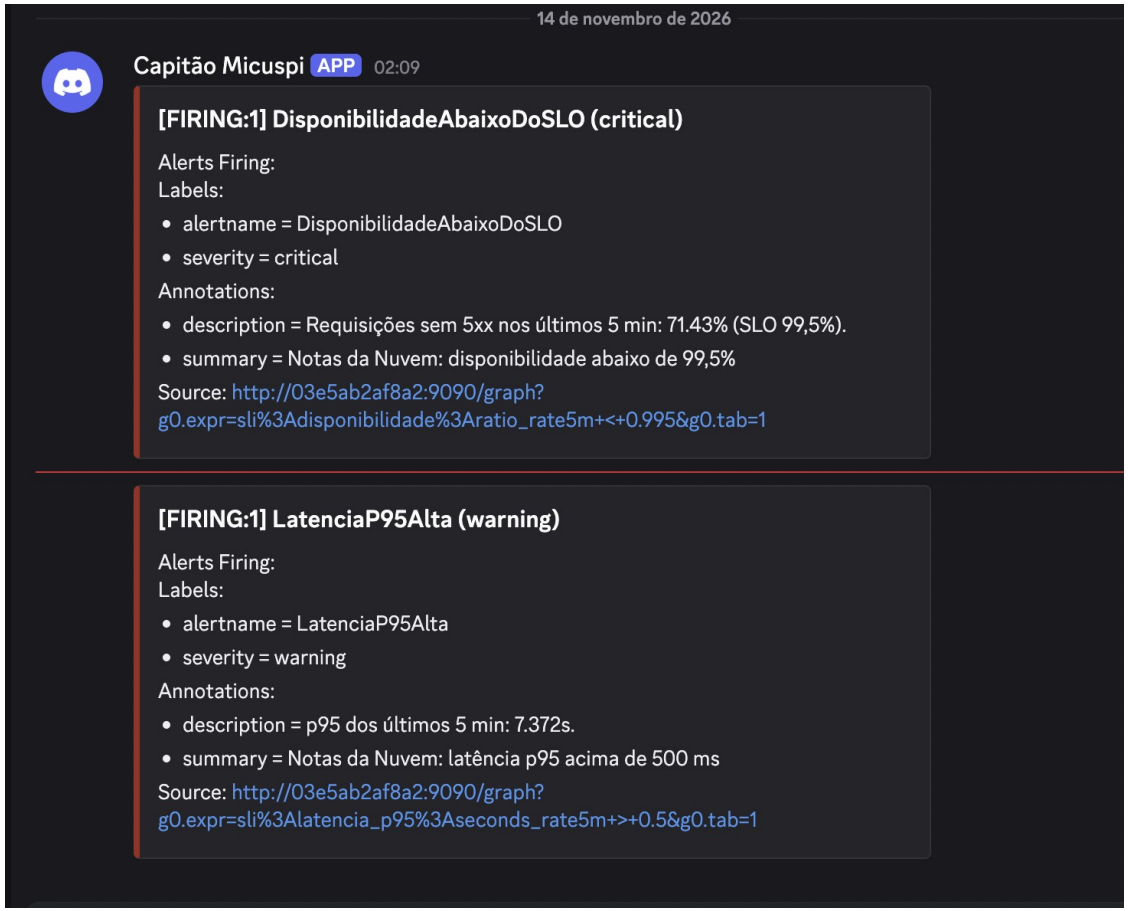
E11-b — Contact points do Alertmanager externo: o receiver `discord`.

Evidência 12 — Alerta disparando e chegando no Discord

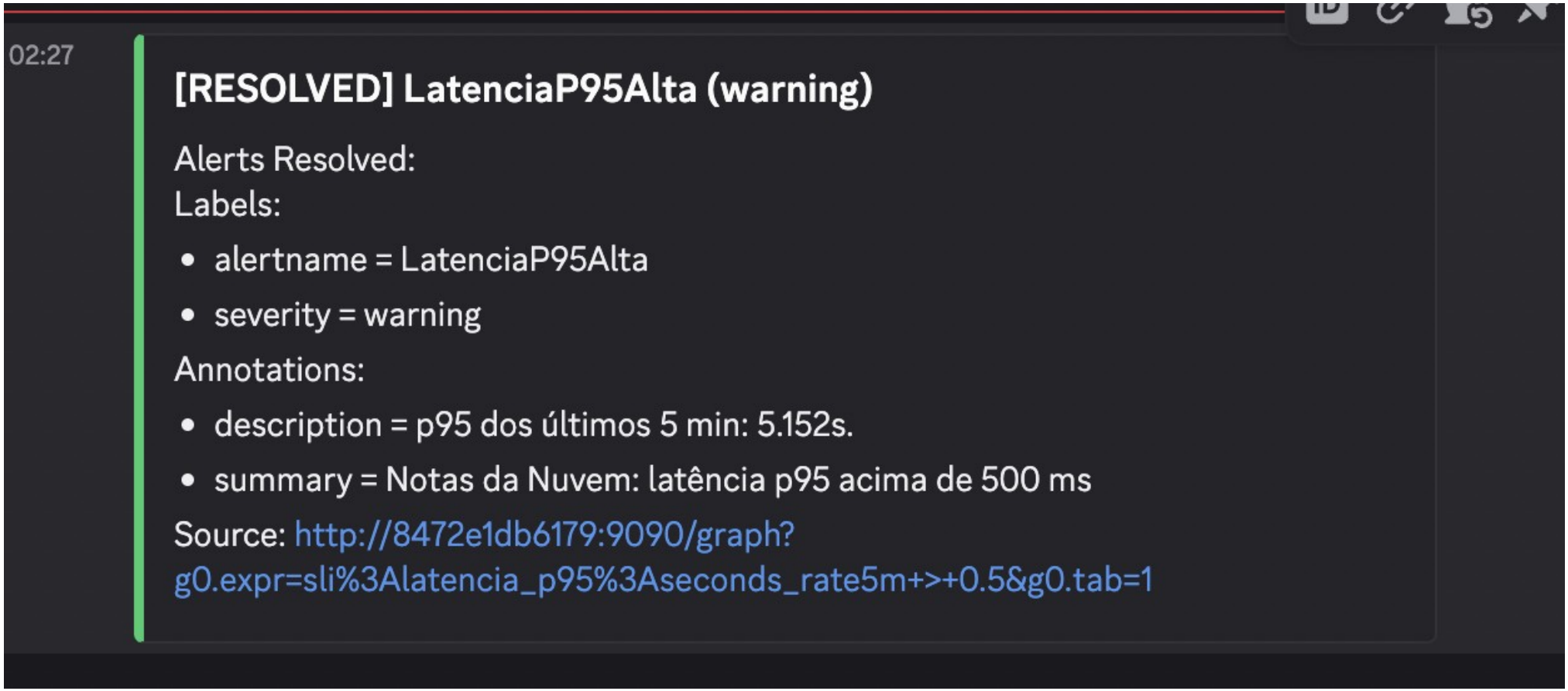
Durante a falha, a `DisponibilidadeAbaixoDoSLO` (critical) e a `LatenciaP95Alta` (warning) passaram para Firing. O Alertmanager entregou as duas no canal do Discord, com o valor medido: 71,43% de disponibilidade e p95 de 7,372 s. Depois que a regra do banco voltou, chegou a notificação RESOLVED. Como a subnet privada não tem NAT, o Alertmanager sai para o Discord por um proxy Squid na EC2 do Grafana, na porta 3128, liberada só para o sg obs-backend (regra proxy_from_backend, listada no log do destroy).



E12-a — Grafana: `DisponibilidadeAbaixoDoSLO` e `LatenciaP95Alta` em Firing (2 firing, 1 normal).



E12-c — Discord: `[FIRING:1] DisponibilidadeAbaixoDoSLO (critical)` e `[FIRING:1] LatenciaP95Alta (warning)`.

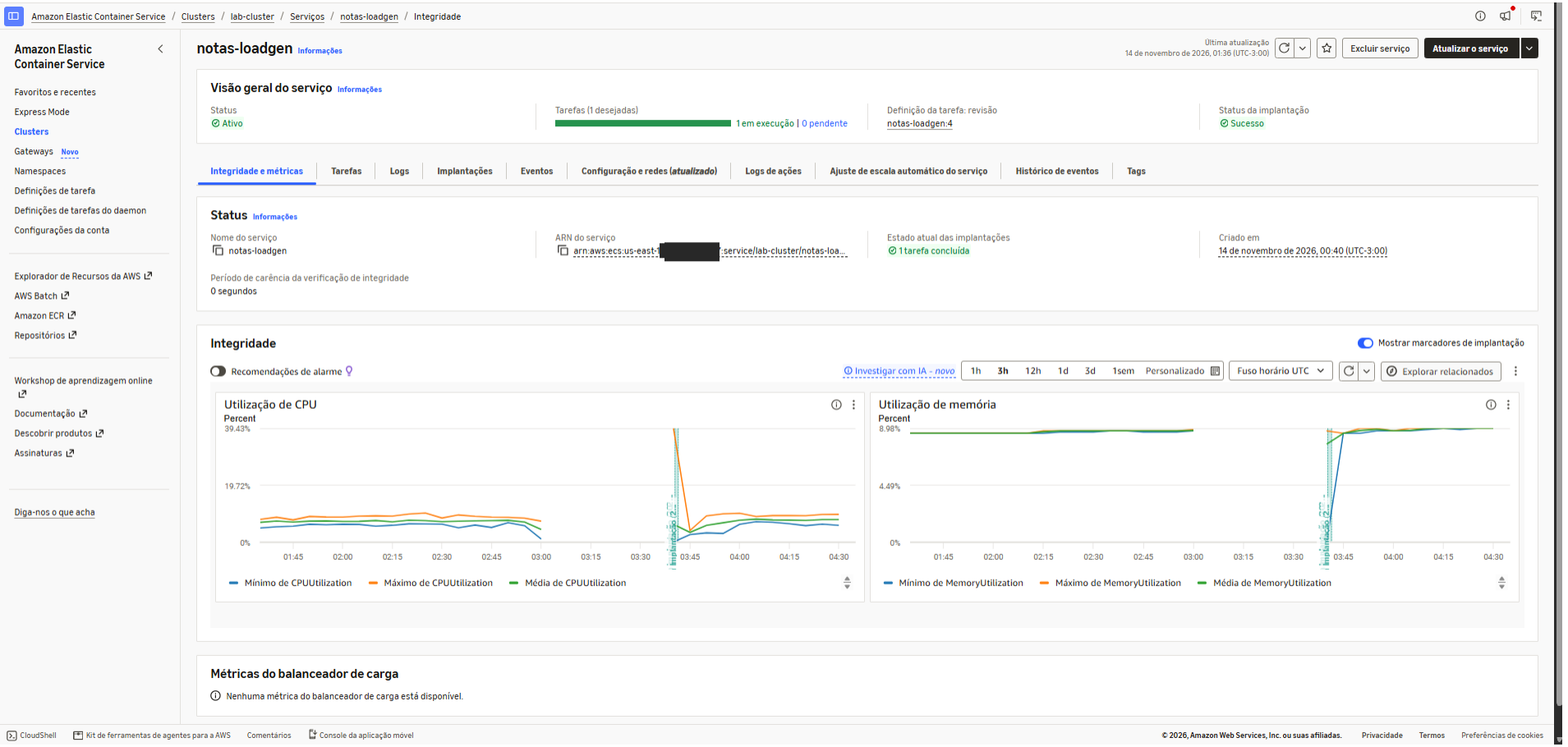


E12-d — Discord: `[RESOLVED] LatenciaP95Alta (warning)` após a recuperação.

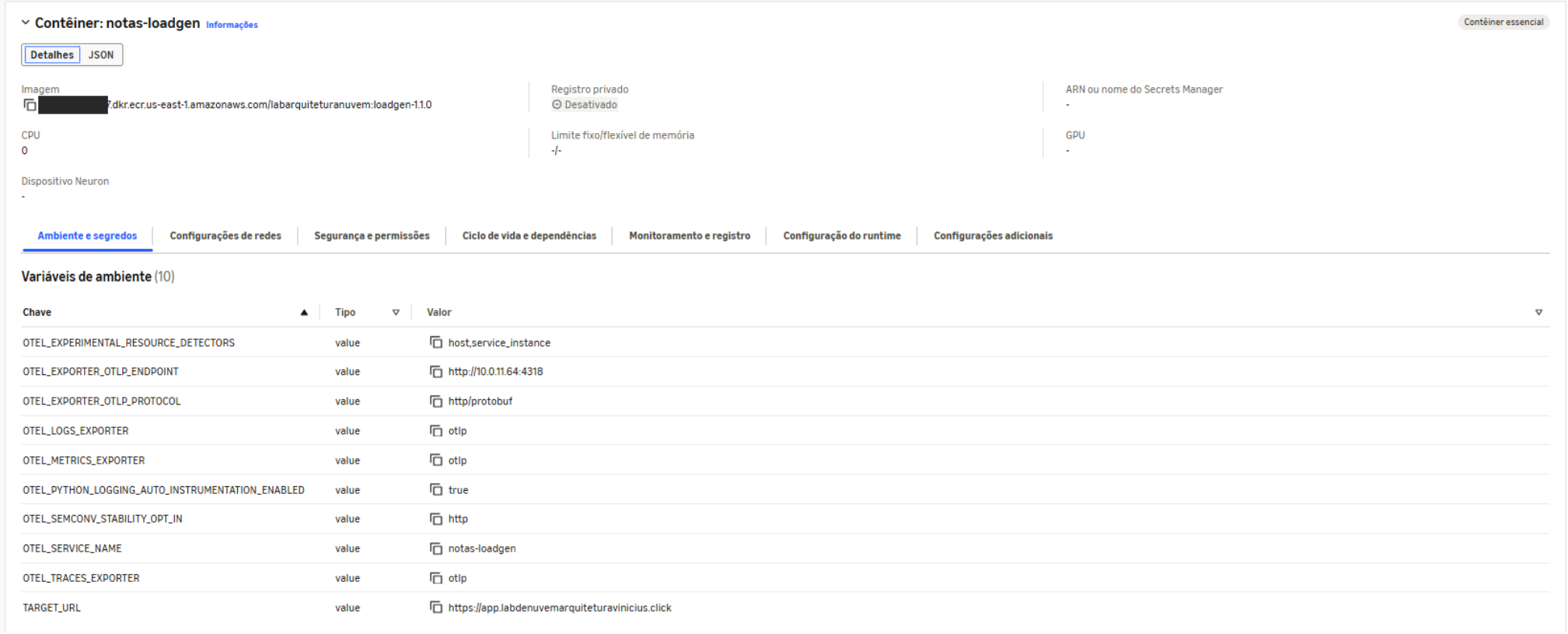
Evidência 13 — Tasks geradoras de tráfego

O service `notas-loadgen` (revisão `notas-loadgen:4`, imagem `loadgen-1.1.0`) roda 1 task no mesmo cluster. Ela chama `https://app.labdenuvemarquiteturavinicius.click`, então cada requisição percorre o caminho completo pela borda, como pede o Passo 6. O loadgen também é instrumentado (`OTEL_SERVICE_NAME=notas-loadgen`): por isso os traces começam nele e continuam na aplicação.

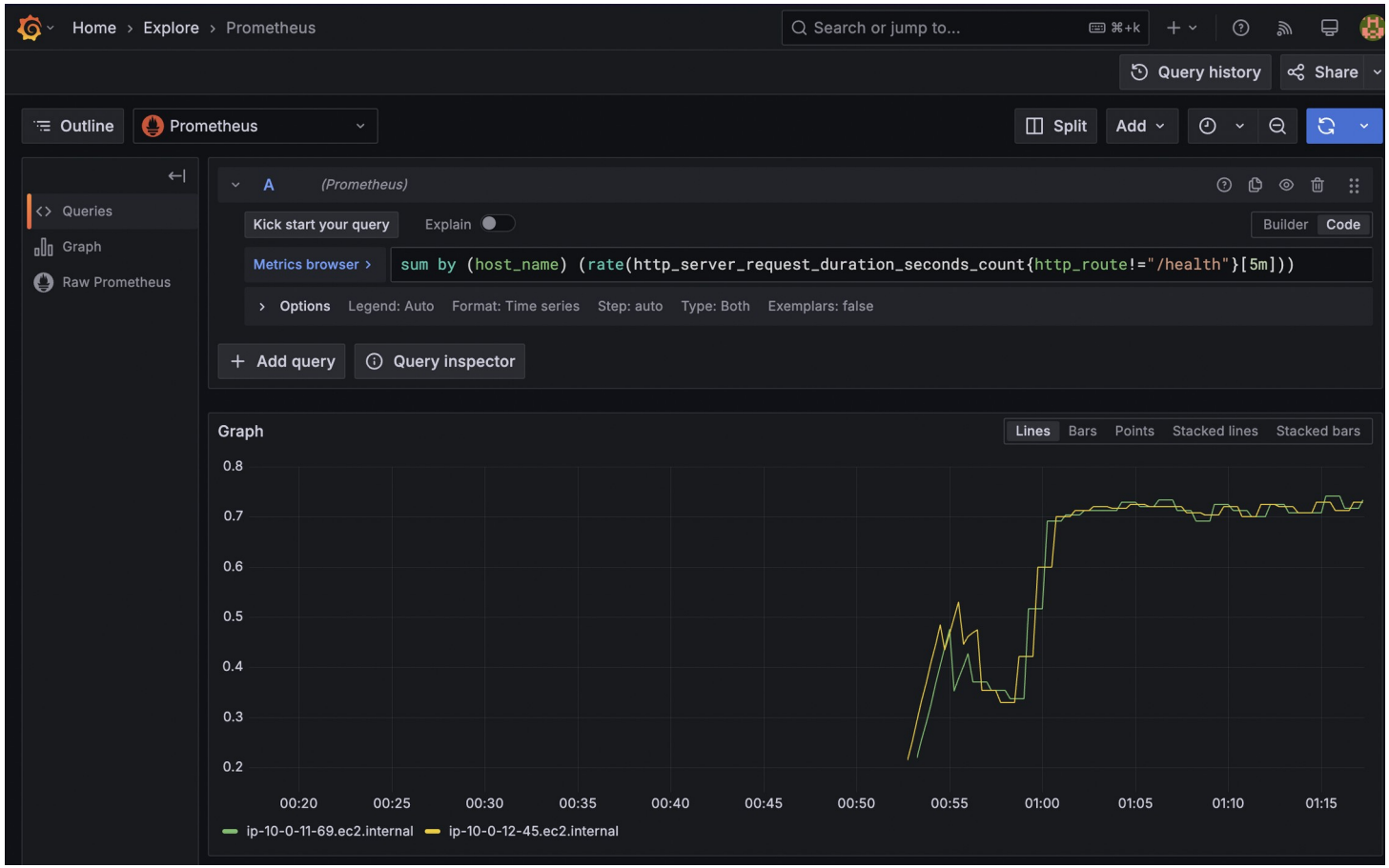
O tráfego fica dividido entre as duas tasks: cerca de 0,7 requisição por segundo em cada uma, sem contar o `/health`.



E13-a — Service `notas-loadgen` ativo, com 1 tarefa em execução.



E13-b — Container `notas-loadgen`: imagem `loadgen-1.1.0`, variáveis `OTEL_*` e `TARGET_URL`.



E13-c — Requisições por segundo por task (`sum by (host_name)`): duas séries, uma por task.

SLI, SLO e regras de alerta

O grupo mediu o SLI dentro da aplicação (opção C da dinâmica da Seção 7). Para cobrir o ponto cego dessa opção, criou um alerta de telemetria ausente.

SLI de disponibilidade, escrito por extenso: proporção das requisições HTTP recebidas pela Notas da Nuvem, exceto `/health`, cujo status não é 5xx. A medição é feita dentro da aplicação pela instrumentação OpenTelemetry do Flask (métrica `http_server_request_duration_seconds`, rótulo `http_response_status_code`), numa janela móvel de 5 minutos.

SLO: 99,5% em 28 dias, o que dá um error budget de cerca de 3 h 22 min de indisponibilidade no período. **SLI de latência:** p95 da mesma métrica em 5 minutos, com alerta acima de 500 ms.

O `/health` fica fora do SLI. Ele é o health check do ALB e responde 200 mesmo com o banco fora, o que diluiria a queda.

Onde medir: por que dentro da aplicação

Opção	Pane total da aplicação aparece no SLI?	Explica a causa?	Decisão
A — navegador	Sim	Não	Descartada: ruidosa e coleta dado do usuário
B — load balancer	Sim	Não	Complementar
C — dentro da aplicação	Não, compensado pelo alerta `TelemetriaAusente`	Sim: rota, consulta, erro exato e trace	**Escolhida**

A evidência 6 mostra a vantagem na prática. Durante a falha do banco, o ALB manteve as duas tasks Healthy (E06-e), enquanto a aplicação registrou cada `POST /notas` com erro, o trace com o timeout e o log com o `OperationalError`.

Regras no Prometheus

As regras ficam no grupo `notas-da-nuvem-sli` do arquivo `alert-rules.yml`. As recording rules pré-calculam o SLI, e o dashboard e os alertas consultam o mesmo valor.

Regra	Tipo	Condição	Severidade	Comportamento na falha
`sli:disponibilidade:ratio_rate5m`	Recording	Proporção sem 5xx	—	Caiu até 58,33%

Regra	Tipo	Condição	Severidade	Comportamento na falha
		em 5 min		
`sli:latencia_p95:seconds_rate5m`	Recording	p95 da duração em 5 min	—	Subiu até 7,37 s
`DisponibilidadeAbaixoDoSLO`	Alerta	`< 0.995` por 1 min	critical	Firing, entregue no Discord
`LatenciaP95Alta`	Alerta	p95 `> 0.5` s	warning	Firing e depois Resolved, no Discord
`TelemetriaAusente`	Alerta	Nenhuma métrica chegando há 10 min	—	Normal (a telemetria continuou chegando)

Decisões de arquitetura e segurança da telemetria

Cada mudança em relação ao roteiro da apostila resolve um requisito extra ou um problema encontrado no caminho.

Decisão	Motivo	Custo ou trade-off
Stack em duas EC2 em vez do `grafana/otel-lgtm` único	Expor só o Grafana; coleta e armazenamento ficam na subnet privada	Uma EC2 t3.small a mais (cerca de US$ 0,02/h)
CloudFront do Grafana removido	O requisito de porta 3000 restrita a um IP não é compatível com origem CloudFront	O Grafana ficou em HTTP puro; HTTPS exigiria CloudFront + WAF com regra de IP
Imagens espelhadas no ECR, em vez de criar um NAT Gateway	A VPC já tinha endpoints de ECR e S3; NAT custa cerca de US$ 0,045/h + tráfego	3 endpoints de SSM a mais para o Session Manager
Uma role IAM por instância	Menor privilégio: só o backend lê o webhook do Discord, só o Grafana lê a senha do admin	—
Regras de alerta no Prometheus, não no Grafana	Versionadas no repositório junto com a infraestrutura	O Grafana só exibe; editar exige um novo deploy
1 worker do gunicorn por task, com 8 threads	Com 2 workers, os dois processos exportavam métricas com o mesmo recurso, e as séries se misturavam no Prometheus	Menos paralelismo de processo, compensado pelas threads
Timeout do health check do ALB em 10 s	Com o banco fora, `/health` leva cerca de 5 s; com o padrão de 5 s, o ALB derrubaria as tasks	Detecção de task travada um pouco mais lenta

Segurança da telemetria. Os quatro cuidados da Seção 10 da apostila foram aplicados:

- **Vazamento:** a consulta `INSERT` usa parâmetro (`%s`), e o texto da nota não aparece nos spans (E04-b). A aplicação não registra o corpo das requisições.
- **Acesso aos painéis:** o Grafana só aceita conexões de um IP `/32` (E09-a), com senha aleatória guardada no Secrets Manager.
- **Endpoints OTLP fechados:** a porta 4318 só aceita o sg das tasks e não tem IP público (E08-c, E09-b).
- **Segredos fora do código:** a senha do banco, o webhook do Discord e a senha do Grafana ficam no Secrets Manager, lidos pelas roles.

No sentido inverso, a mesma telemetria serve à defesa: o painel e os alertas mostram picos de 5xx e de latência como os da falha provocada.